

Universidad Tecnológica de El Salvador



FACULTAD DE INFORMATICA Y CIENCIAS APLICADAS ESCUELA DE CIENCIAS APLICADAS

GRUPO 12

INTEGRANTES:

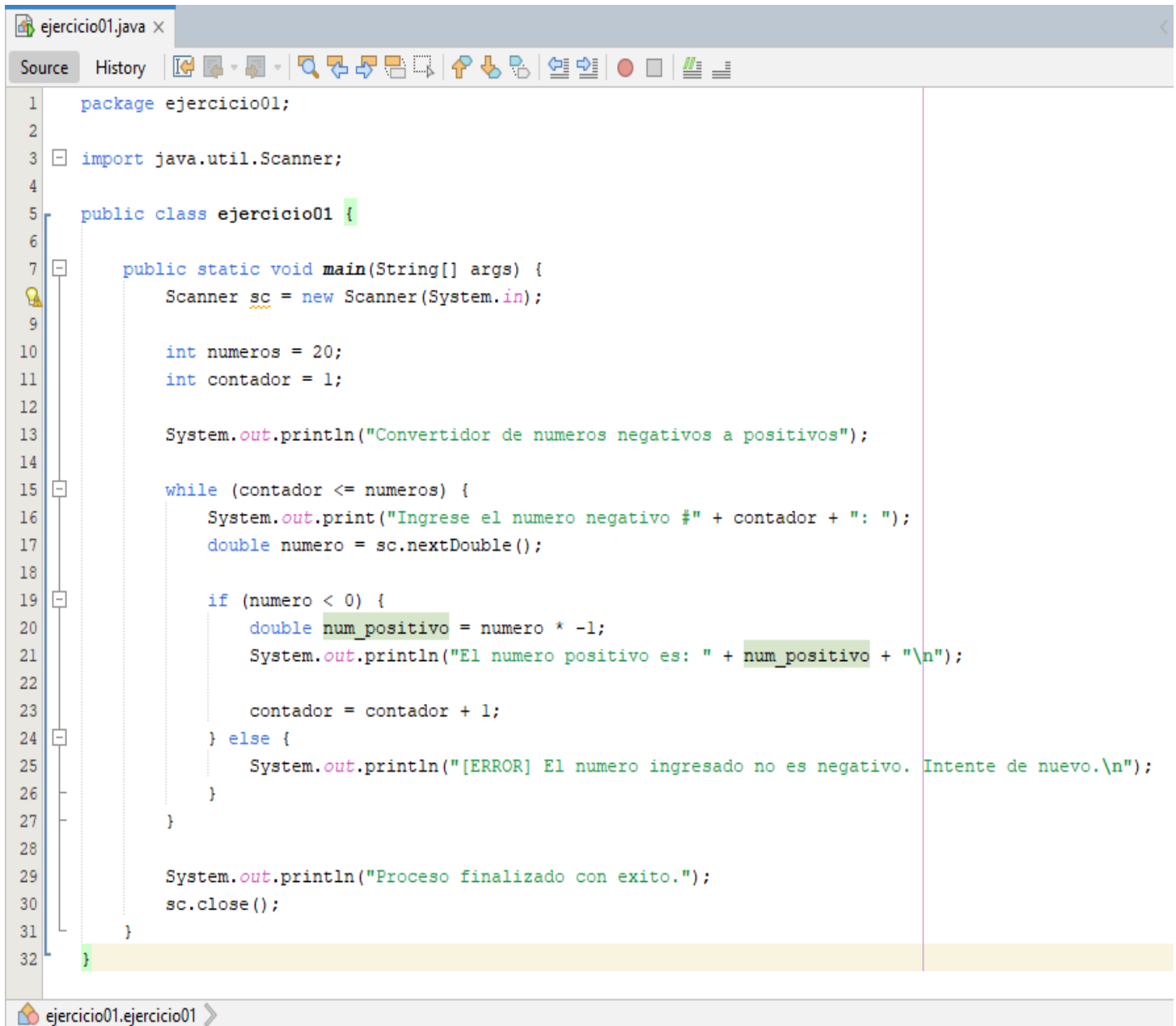
<i>ERIK ARMANDO MAYEN LOPEZ</i>	CARNET: 29-1001-2026
<i>GABRIELA NOEMY FLORES PINEDA</i>	CARNET: 29-1939-2026
<i>BRANDON XAVIER LÓPEZ BERNAL</i>	CARNET: 29-3018-2026
<i>DENIS HERNANDEZ</i>	CARNET: 29-2476-2026

NOMBRE DE DOCENTE: ING. SANDRA MARITZA CANTARERO

TEMA: ESTRUCTURA REPETITIVA MIENTRAS Y HACER MIENTRAS

1- Leer 20 números negativos, convertirlos a positivos e imprimir dichos números.

Archivo de código (NetBeans)



```
1 package ejercicio01;
2
3 import java.util.Scanner;
4
5 public class ejercicio01 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9
10        int numeros = 20;
11        int contador = 1;
12
13        System.out.println("Convertidor de numeros negativos a positivos");
14
15        while (contador <= numeros) {
16            System.out.print("Ingrese el numero negativo #" + contador + ": ");
17            double numero = sc.nextDouble();
18
19            if (numero < 0) {
20                double num_positivo = numero * -1;
21                System.out.println("El numero positivo es: " + num_positivo + "\n");
22
23                contador = contador + 1;
24            } else {
25                System.out.println("[ERROR] El numero ingresado no es negativo. Intente de nuevo.\n");
26            }
27        }
28
29        System.out.println("Proceso finalizado con exito.");
30        sc.close();
31    }
32 }
```

Consola de salida (Ejercicio 1)

```
Output - EjerciciosGuiaPOO (run)

Ingrese el numero negativo #13: 267
[ERROR] El numero ingresado no es negativo. Intente de nuevo.

Ingrese el numero negativo #13: -267
El numero positivo es: 267.0

Ingrese el numero negativo #14: -1279
El numero positivo es: 1279.0

Ingrese el numero negativo #15: -596
El numero positivo es: 596.0

Ingrese el numero negativo #16: 746
[ERROR] El numero ingresado no es negativo. Intente de nuevo.

Ingrese el numero negativo #16: -746
El numero positivo es: 746.0

Ingrese el numero negativo #17: 247
[ERROR] El numero ingresado no es negativo. Intente de nuevo.

Ingrese el numero negativo #17: -247
El numero positivo es: 247.0

Ingrese el numero negativo #18: -915
El numero positivo es: 915.0

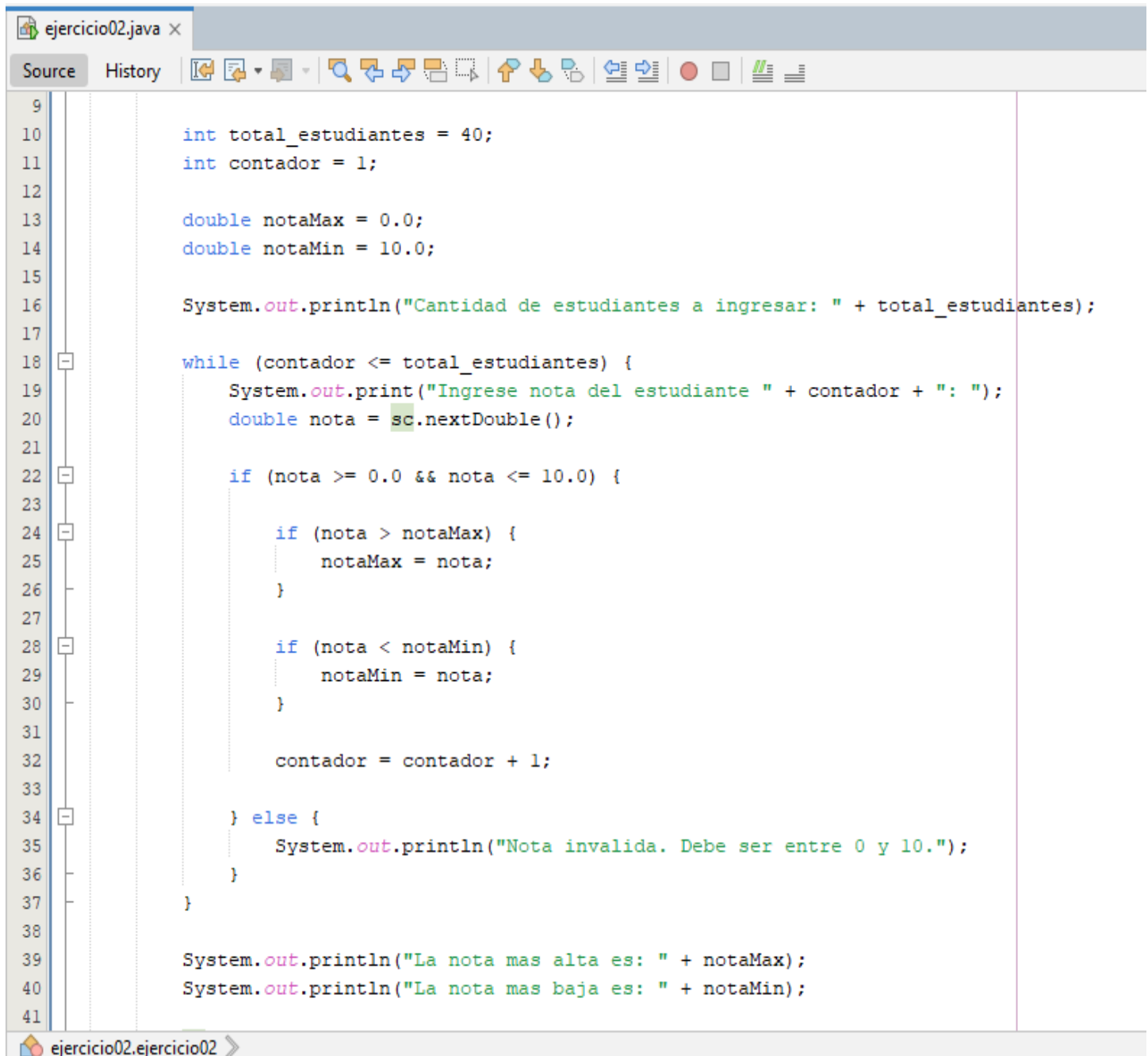
Ingrese el numero negativo #19: -1739
El numero positivo es: 1739.0

Ingrese el numero negativo #20: -5894
El numero positivo es: 5894.0

Proceso finalizado con exito.
BUILD SUCCESSFUL (total time: 4 minutes 54 seconds)
```

2- Calcule e imprima la calificación más alta y la calificación más baja de 40 estudiantes.

Archivo de código (NetBeans)



```
9
10     int total_estudiantes = 40;
11     int contador = 1;
12
13     double notaMax = 0.0;
14     double notaMin = 10.0;
15
16     System.out.println("Cantidad de estudiantes a ingresar: " + total_estudiantes);
17
18     while (contador <= total_estudiantes) {
19         System.out.print("Ingrese nota del estudiante " + contador + ": ");
20         double nota = sc.nextDouble();
21
22         if (nota >= 0.0 && nota <= 10.0) {
23
24             if (nota > notaMax) {
25                 notaMax = nota;
26             }
27
28             if (nota < notaMin) {
29                 notaMin = nota;
30             }
31
32             contador = contador + 1;
33
34         } else {
35             System.out.println("Nota invalida. Debe ser entre 0 y 10.");
36         }
37     }
38
39     System.out.println("La nota mas alta es: " + notaMax);
40     System.out.println("La nota mas baja es: " + notaMin);
41
```

Consola de salida (Ejercicio 2)

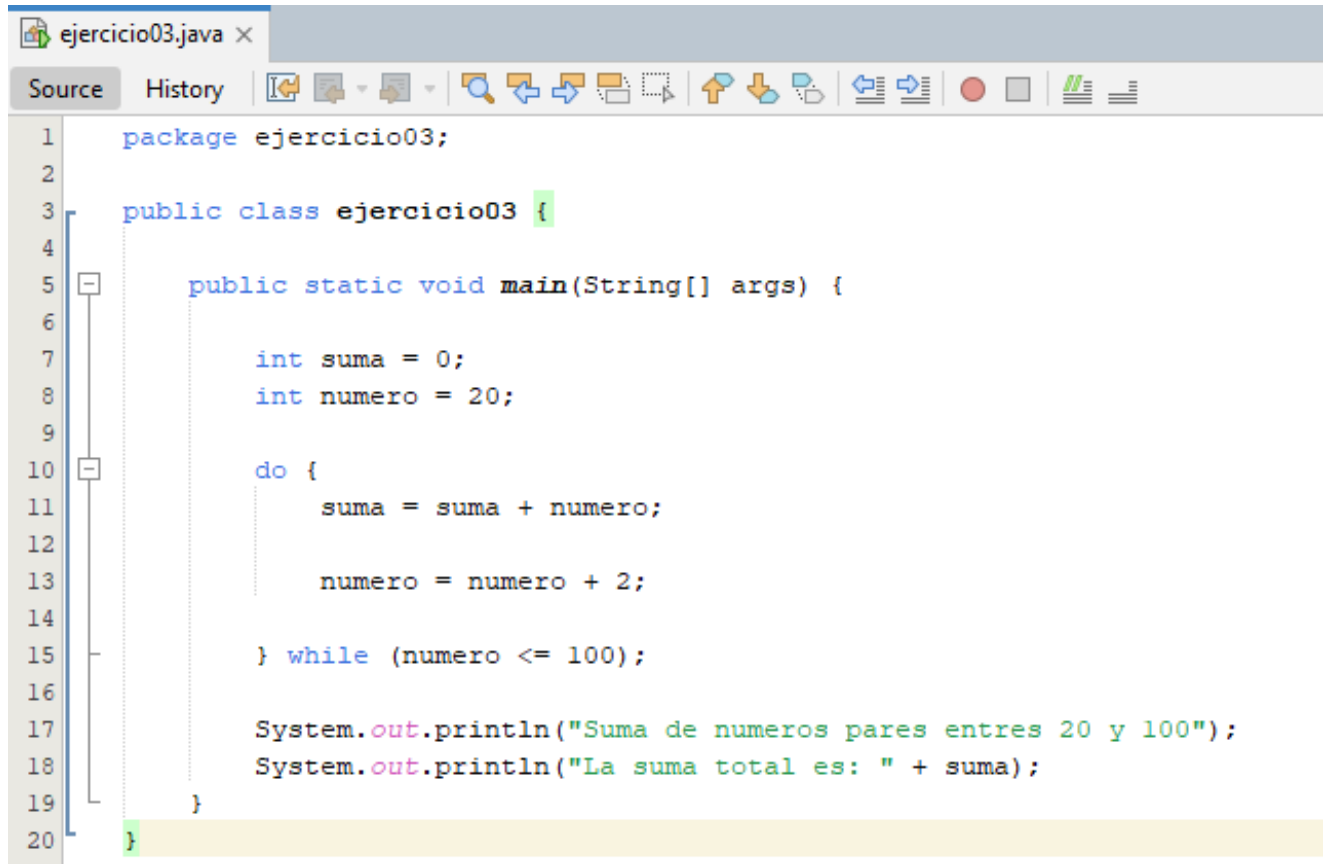
Output - EjerciciosGuiaPOO (run)



```
Ingrese nota del estudiante 8: 4
Ingrese nota del estudiante 9: 8
Ingrese nota del estudiante 10: 6
Ingrese nota del estudiante 11: 9
Ingrese nota del estudiante 12: 10
Ingrese nota del estudiante 13: 3
Ingrese nota del estudiante 14: 4
Ingrese nota del estudiante 15: 6
Ingrese nota del estudiante 16: 7
Ingrese nota del estudiante 17: 8
Ingrese nota del estudiante 18: 6
Ingrese nota del estudiante 19: 9
Ingrese nota del estudiante 20: 6
Ingrese nota del estudiante 21: 7
Ingrese nota del estudiante 22: 8
Ingrese nota del estudiante 23: 9
Ingrese nota del estudiante 24: 6
Ingrese nota del estudiante 25: 9
Ingrese nota del estudiante 26: 4
Ingrese nota del estudiante 27: 6
Ingrese nota del estudiante 28: 7
Ingrese nota del estudiante 29: 2
Ingrese nota del estudiante 30: 9
Ingrese nota del estudiante 31: 10
Ingrese nota del estudiante 32: 9
Ingrese nota del estudiante 33: 7
Ingrese nota del estudiante 34: 8
Ingrese nota del estudiante 35: 9
Ingrese nota del estudiante 36: 6
Ingrese nota del estudiante 37: 9
Ingrese nota del estudiante 38: 10
Ingrese nota del estudiante 39: 7
Ingrese nota del estudiante 40: 8
La nota mas alta es: 10.0
La nota mas baja es: 2.0
BUILD SUCCESSFUL (total time: 1 minute 1 second)
```

3- Calcule e imprima la suma de los números pares comprendidos entre 20 y 100.

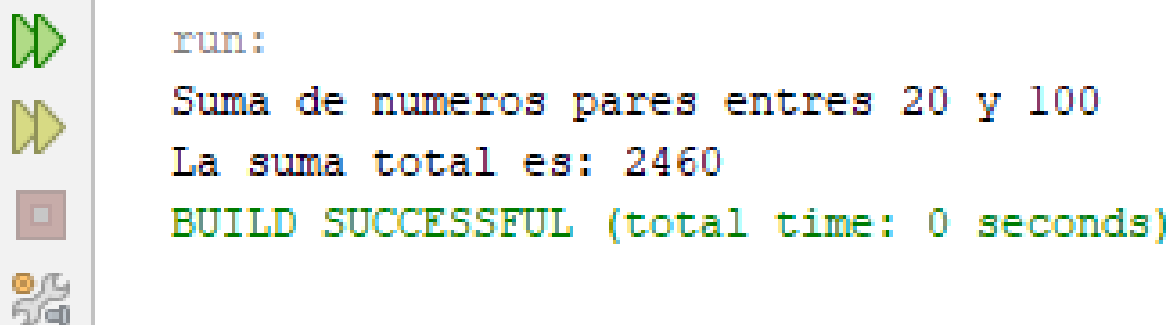
Archivo de código (NetBeans)



```
1 package ejercicio03;
2
3 public class ejercicio03 {
4
5     public static void main(String[] args) {
6
7         int suma = 0;
8         int numero = 20;
9
10        do {
11            suma = suma + numero;
12
13            numero = numero + 2;
14
15        } while (numero <= 100);
16
17        System.out.println("Suma de numeros pares entres 20 y 100");
18        System.out.println("La suma total es: " + suma);
19    }
20 }
```

Consola de salida (Ejercicio 3)

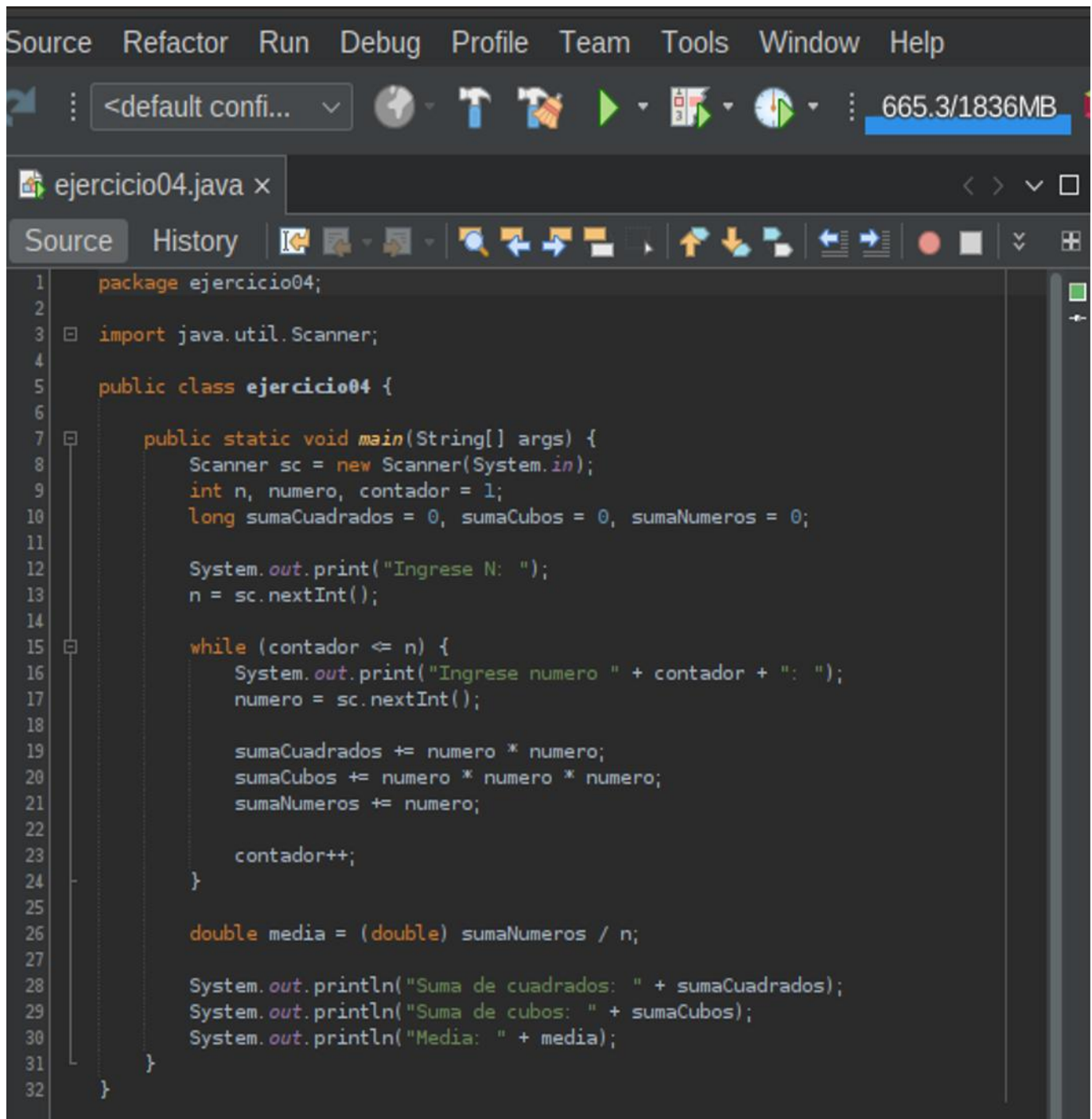
Output - EjerciciosGuiaPOO (run)



```
run:
Suma de numeros pares entres 20 y 100
La suma total es: 2460
BUILD SUCCESSFUL (total time: 0 seconds)
```

4- Calcule e imprima la suma de los cuadrados y cubos de N números enteros positivos. Además, calcule la media de los n números positivos.

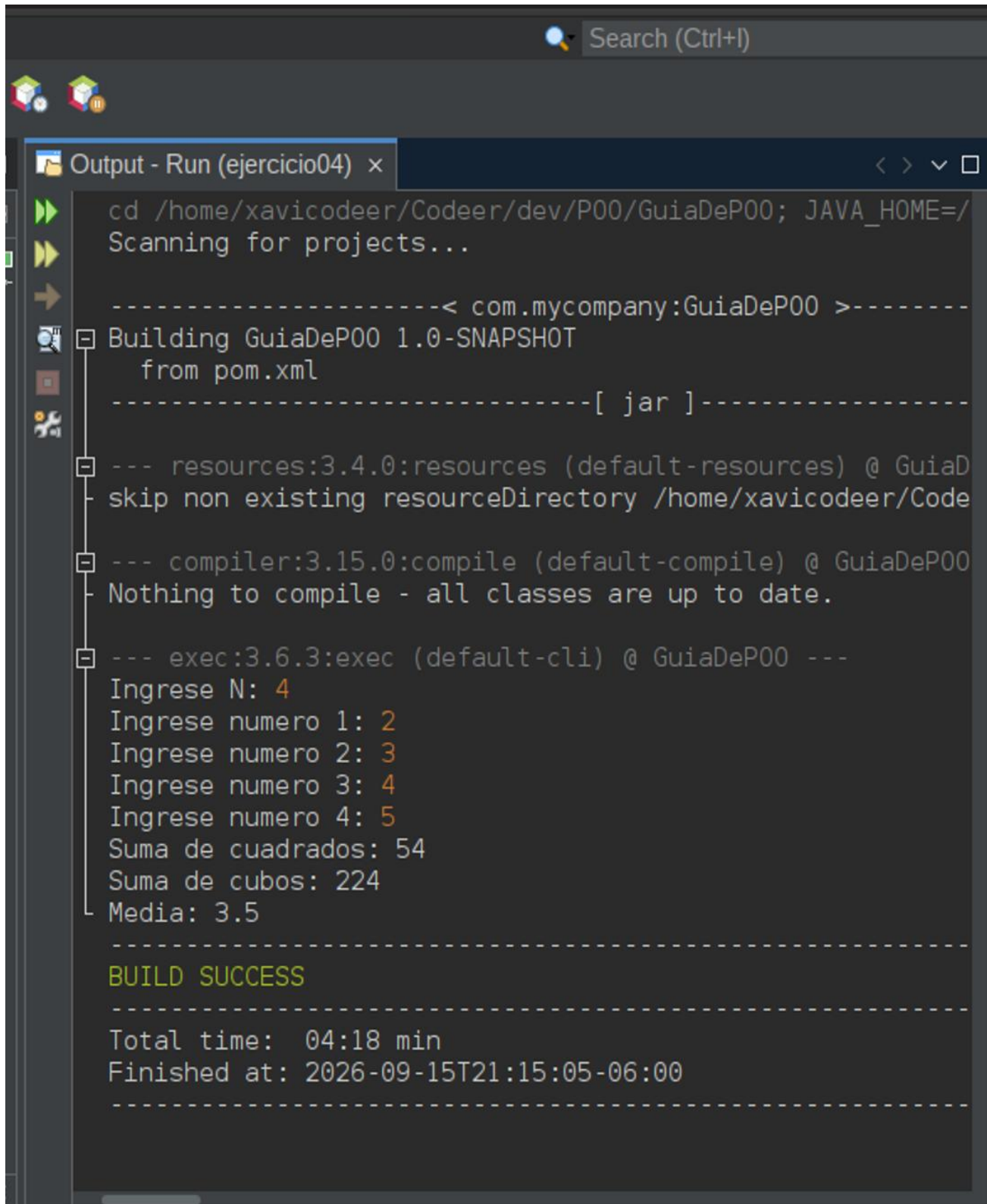
[Archivo de código \(NetBeans\)](#)



The screenshot displays the NetBeans IDE interface. The top menu bar includes 'Source', 'Refactor', 'Run', 'Debug', 'Profile', 'Team', 'Tools', 'Window', and 'Help'. Below the menu is a toolbar with various icons, including a configuration dropdown set to '<default confi...', a globe icon, a wrench icon, a green play button, a red stop button, and a memory usage indicator showing '665.3/1836MB'. The main editor window is titled 'ejercicio04.java' and shows the following Java code:

```
1 package ejercicio04;
2
3 import java.util.Scanner;
4
5 public class ejercicio04 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int n, numero, contador = 1;
10        long sumaCuadrados = 0, sumaCubos = 0, sumaNumeros = 0;
11
12        System.out.print("Ingrese N: ");
13        n = sc.nextInt();
14
15        while (contador <= n) {
16            System.out.print("Ingrese numero " + contador + ": ");
17            numero = sc.nextInt();
18
19            sumaCuadrados += numero * numero;
20            sumaCubos += numero * numero * numero;
21            sumaNumeros += numero;
22
23            contador++;
24        }
25
26        double media = (double) sumaNumeros / n;
27
28        System.out.println("Suma de cuadrados: " + sumaCuadrados);
29        System.out.println("Suma de cubos: " + sumaCubos);
30        System.out.println("Media: " + media);
31    }
32 }
```

Consola de salida (Ejercicio 4)



The screenshot shows an IDE console window titled "Output - Run (ejercicio04)". The console output displays the execution of a Maven build for a project named "GuiaDeP00". The build process includes scanning for projects, building the project from a POM file, and running a series of tests. The tests involve inputting four numbers (2, 3, 4, 5) and calculating the sum of squares (54) and the sum of cubes (224). The build concludes with a "BUILD SUCCESS" message and a total execution time of 04:18 minutes.

```
cd /home/xavicodeer/Codeer/dev/P00/GuiaDeP00; JAVA_HOME=/
Scanning for projects...

-----< com.mycompany:GuiaDeP00 >-----
Building GuiaDeP00 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----

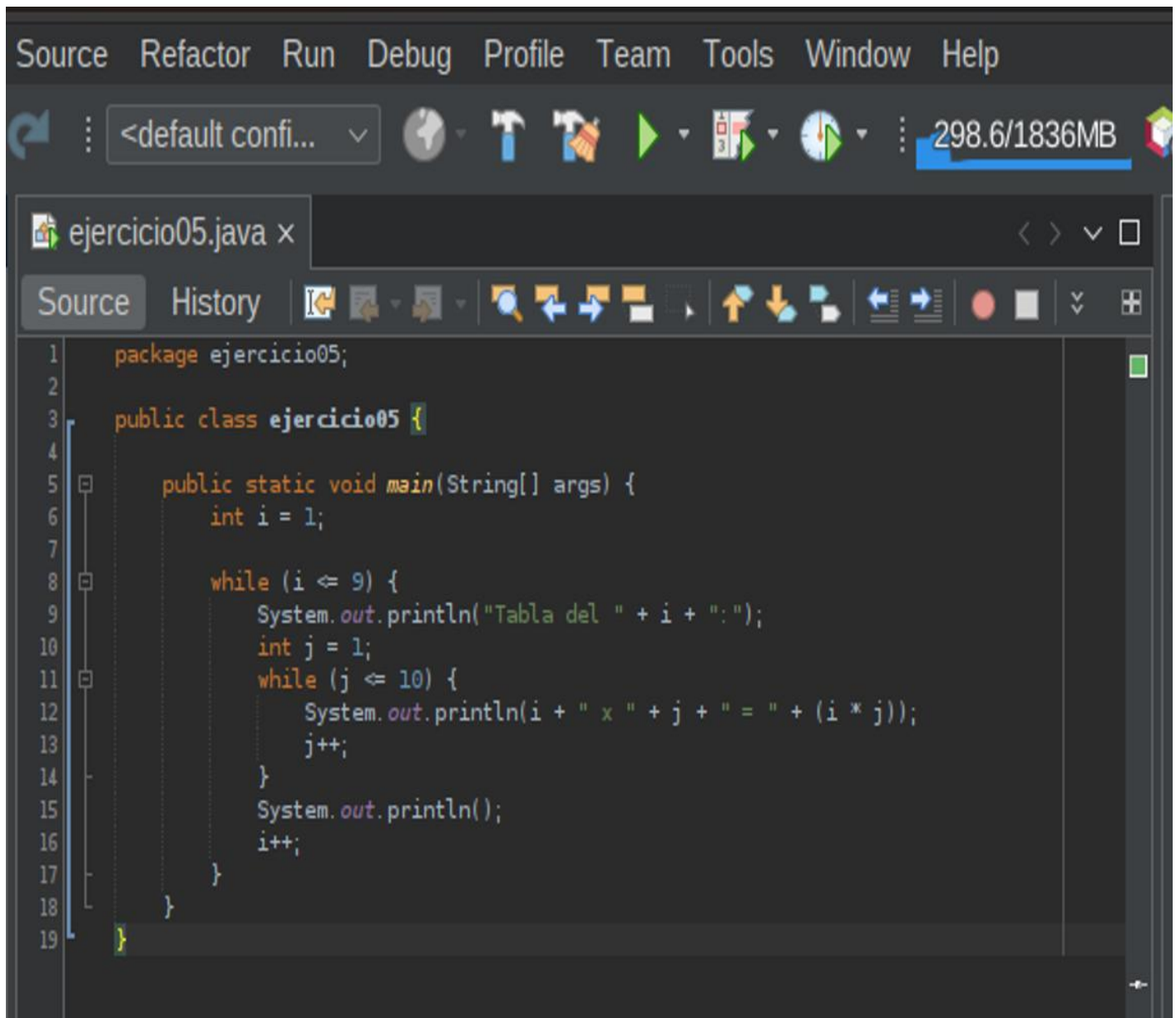
--- resources:3.4.0:resources (default-resources) @ GuiaD
skip non existing resourceDirectory /home/xavicodeer/Code
--- compiler:3.15.0:compile (default-compile) @ GuiaDeP00
Nothing to compile - all classes are up to date.
--- exec:3.6.3:exec (default-cli) @ GuiaDeP00 ---
Ingrese N: 4
Ingrese numero 1: 2
Ingrese numero 2: 3
Ingrese numero 3: 4
Ingrese numero 4: 5
Suma de cuadrados: 54
Suma de cubos: 224
Media: 3.5

BUILD SUCCESS

Total time: 04:18 min
Finished at: 2026-09-15T21:15:05-06:00
```

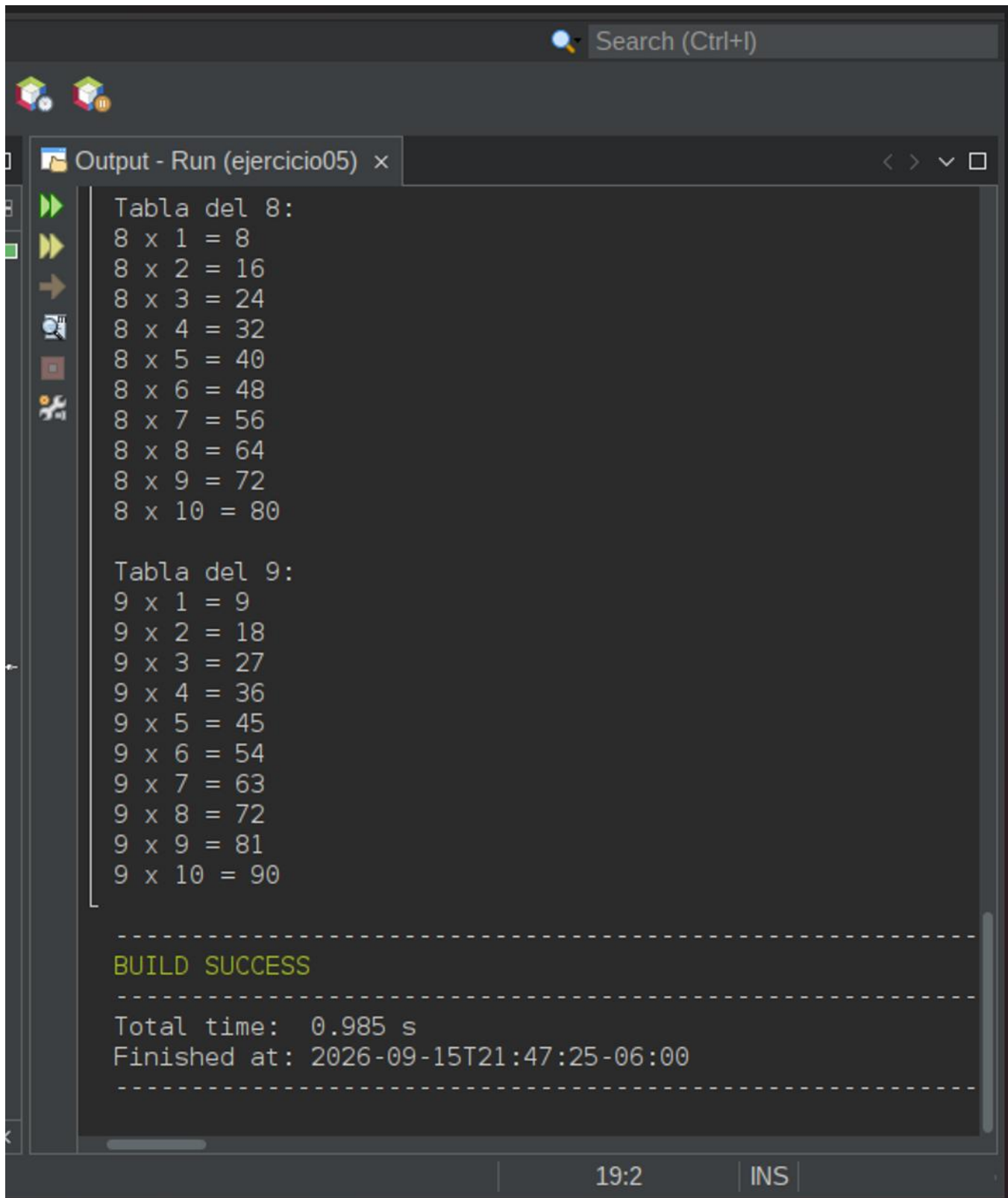
5- Imprima todas las tablas de multiplicar (números 1 al 9).

Archivo de código (NetBeans)



```
1 package ejercicio05;
2
3 public class ejercicio05 {
4
5     public static void main(String[] args) {
6         int i = 1;
7
8         while (i <= 9) {
9             System.out.println("Tabla del " + i + ":");
10            int j = 1;
11            while (j <= 10) {
12                System.out.println(i + " x " + j + " = " + (i * j));
13                j++;
14            }
15            System.out.println();
16            i++;
17        }
18    }
19 }
```

Consola de salida (Ejercicio 5)



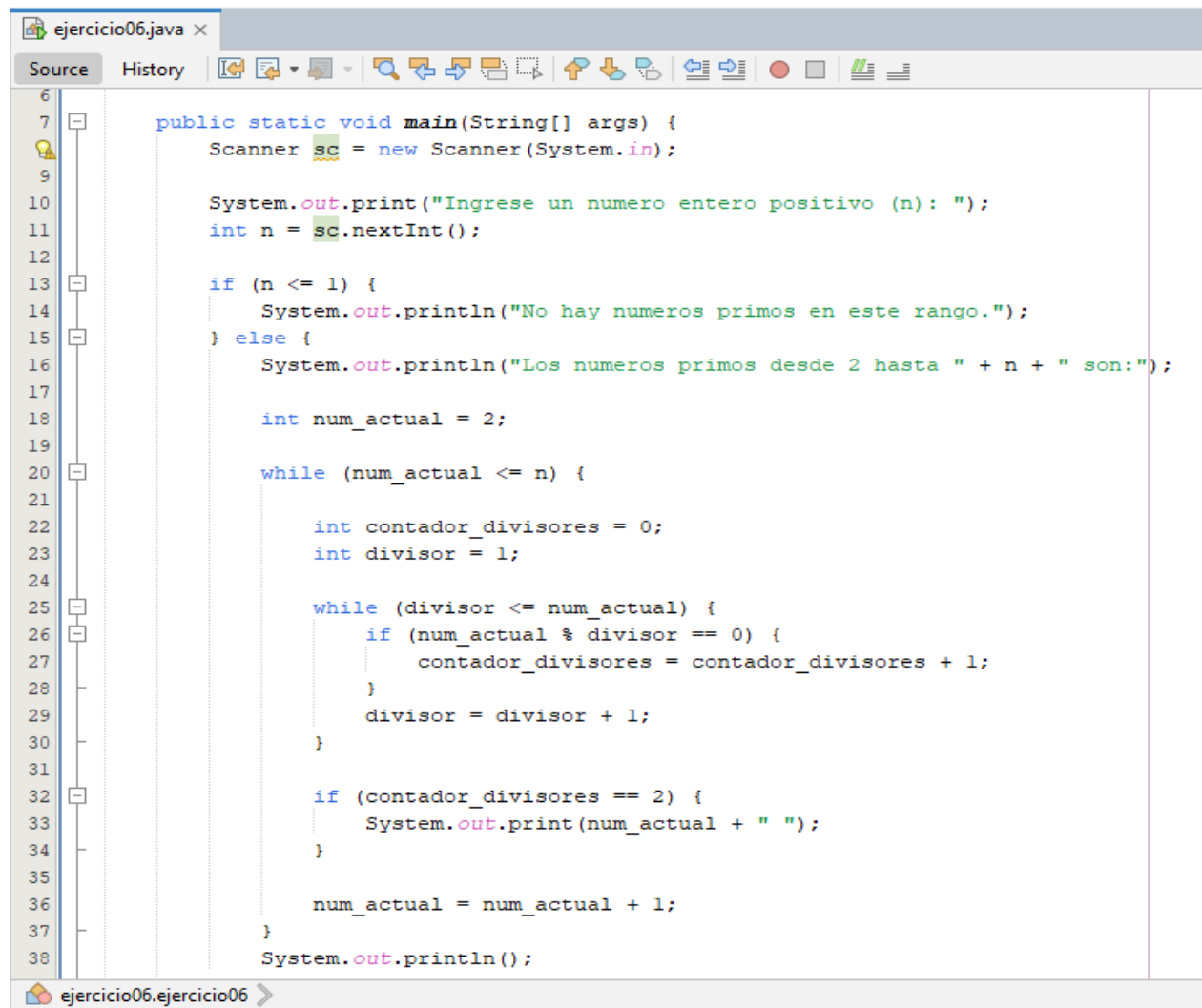
The screenshot shows an IDE's Run Output window. The title bar reads "Output - Run (ejercicio05) x". The console content is as follows:

```
Tabla del 8:  
8 x 1 = 8  
8 x 2 = 16  
8 x 3 = 24  
8 x 4 = 32  
8 x 5 = 40  
8 x 6 = 48  
8 x 7 = 56  
8 x 8 = 64  
8 x 9 = 72  
8 x 10 = 80  
  
Tabla del 9:  
9 x 1 = 9  
9 x 2 = 18  
9 x 3 = 27  
9 x 4 = 36  
9 x 5 = 45  
9 x 6 = 54  
9 x 7 = 63  
9 x 8 = 72  
9 x 9 = 81  
9 x 10 = 90  
  
-----  
BUILD SUCCESS  
-----  
Total time: 0.985 s  
Finished at: 2026-09-15T21:47:25-06:00  
-----
```

At the bottom of the IDE window, the status bar shows "19:2" and "INS".

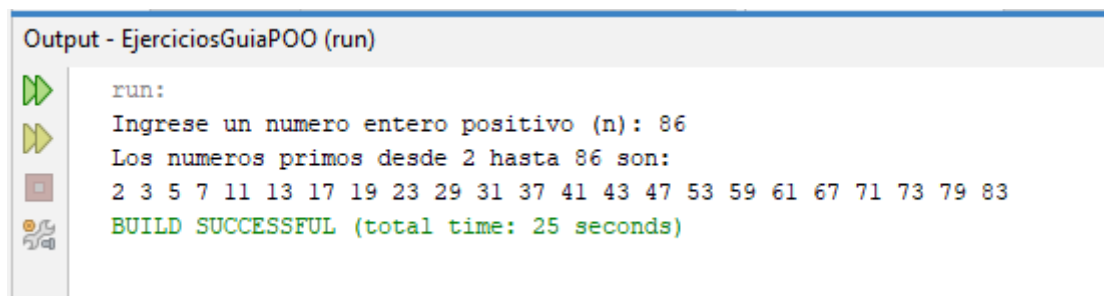
6- Calcule e imprima los números primos de n número enteros positivos.

Archivo de código (NetBeans)



```
6
7 public static void main(String[] args) {
8     Scanner sc = new Scanner(System.in);
9
10    System.out.print("Ingrese un numero entero positivo (n): ");
11    int n = sc.nextInt();
12
13    if (n <= 1) {
14        System.out.println("No hay numeros primos en este rango.");
15    } else {
16        System.out.println("Los numeros primos desde 2 hasta " + n + " son:");
17
18        int num_actual = 2;
19
20        while (num_actual <= n) {
21
22            int contador_divisores = 0;
23            int divisor = 1;
24
25            while (divisor <= num_actual) {
26                if (num_actual % divisor == 0) {
27                    contador_divisores = contador_divisores + 1;
28                }
29                divisor = divisor + 1;
30            }
31
32            if (contador_divisores == 2) {
33                System.out.print(num_actual + " ");
34            }
35
36            num_actual = num_actual + 1;
37        }
38        System.out.println();
39    }
40 }
```

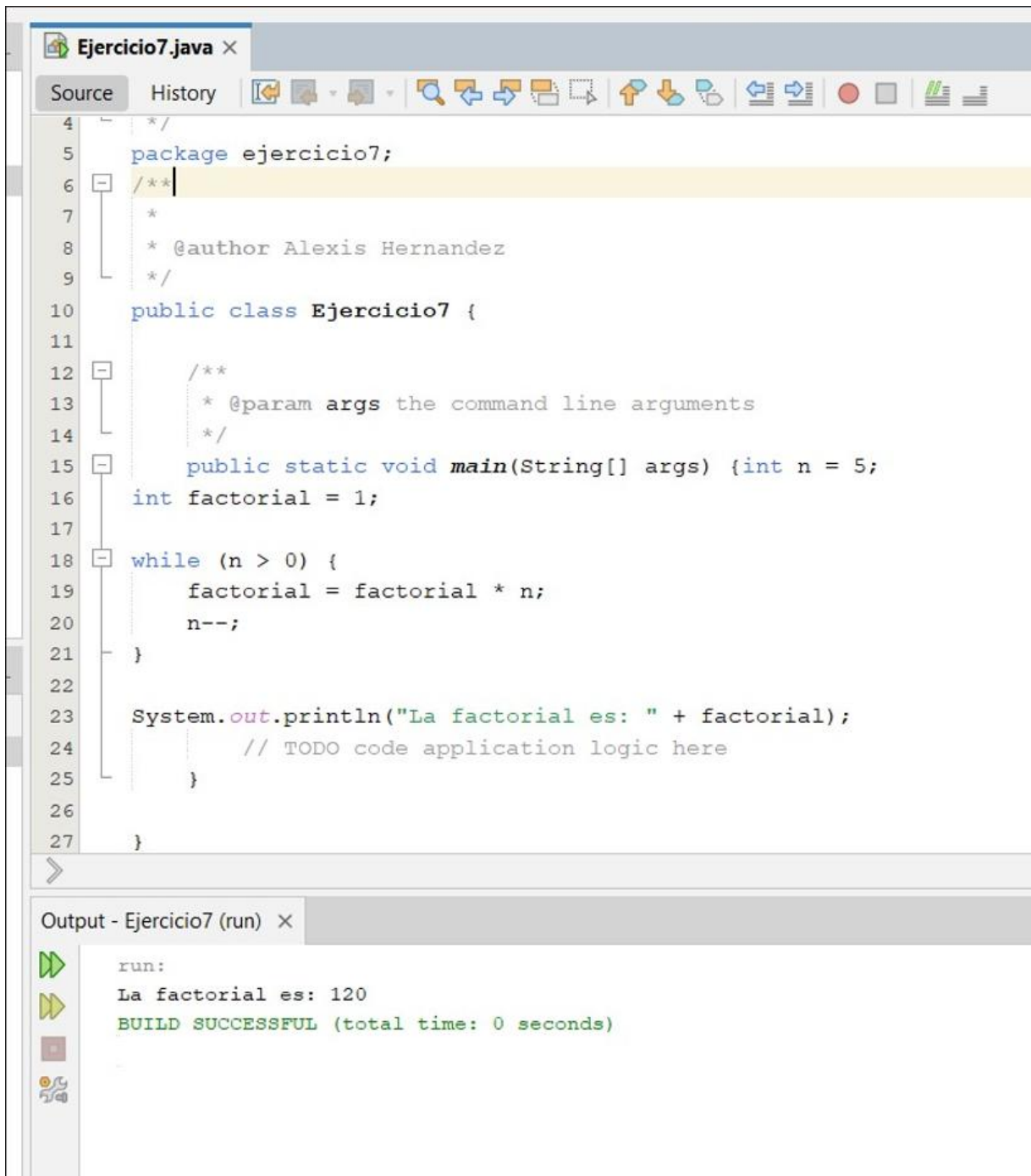
Consola de salida (Ejercicio 6)



```
Output - EjerciciosGuiaPOO (run)
run:
Ingrese un numero entero positivo (n): 86
Los numeros primos desde 2 hasta 86 son:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83
BUILD SUCCESSFUL (total time: 25 seconds)
```

7- Calcular e imprimir la factorial de n número positivos.

Archivo de código (NetBeans) y Consola de salida (Ejercicio 7)



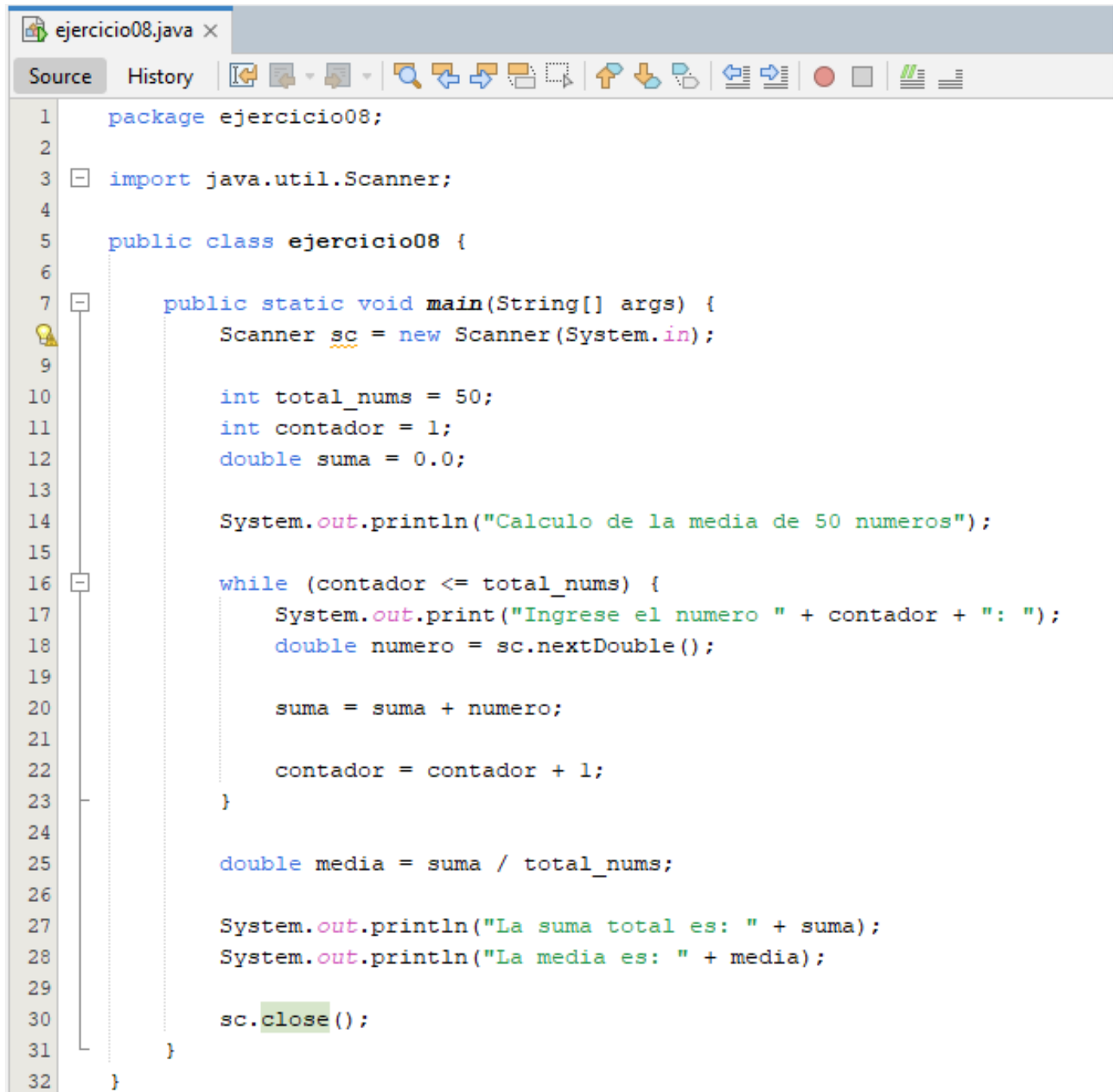
```
4  */
5  package ejercicio7;
6  /**
7   *
8   * @author Alexis Hernandez
9   */
10 public class Ejercicio7 {
11
12     /**
13      * @param args the command line arguments
14      */
15     public static void main(String[] args) {int n = 5;
16     int factorial = 1;
17
18     while (n > 0) {
19         factorial = factorial * n;
20         n--;
21     }
22
23     System.out.println("La factorial es: " + factorial);
24     // TODO code application logic here
25 }
26
27 }
```

Output - Ejercicio7 (run) ×

```
run:
La factorial es: 120
BUILD SUCCESSFUL (total time: 0 seconds)
```

8- Calcular la media de 50 número e imprimir el resultado.

Archivo de código (NetBeans)



```
1 package ejercicio08;
2
3 import java.util.Scanner;
4
5 public class ejercicio08 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9
10        int total_nums = 50;
11        int contador = 1;
12        double suma = 0.0;
13
14        System.out.println("Calculo de la media de 50 numeros");
15
16        while (contador <= total_nums) {
17            System.out.print("Ingrese el numero " + contador + ": ");
18            double numero = sc.nextDouble();
19
20            suma = suma + numero;
21
22            contador = contador + 1;
23        }
24
25        double media = suma / total_nums;
26
27        System.out.println("La suma total es: " + suma);
28        System.out.println("La media es: " + media);
29
30        sc.close();
31    }
32 }
```

Consola de salida (Ejercicio 8)

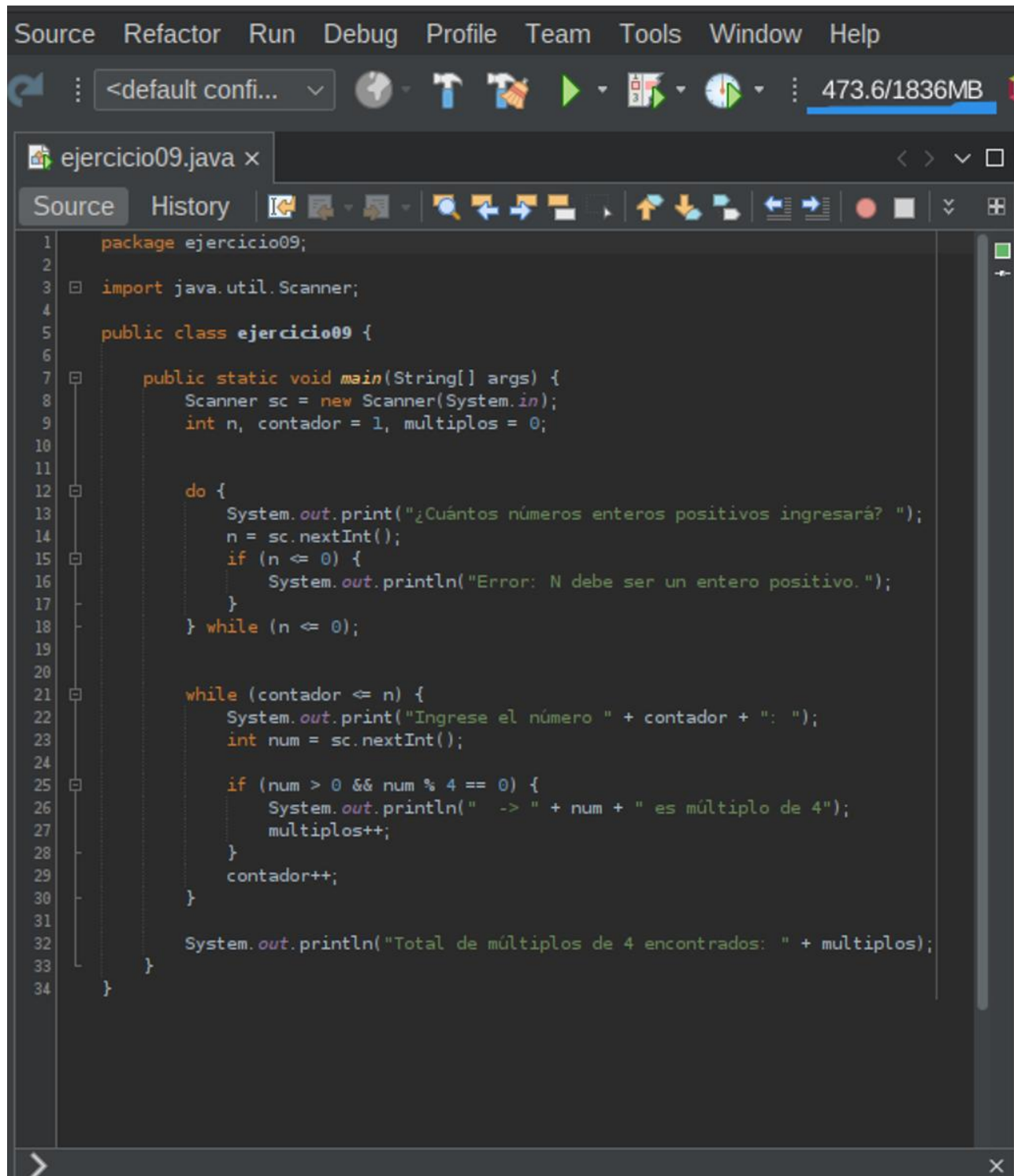
Output - EjerciciosGuiaPOO (run)



```
Ingrese el numero 18: 42
Ingrese el numero 19: 6
Ingrese el numero 20: 9
Ingrese el numero 21: 10
Ingrese el numero 22: 46
Ingrese el numero 23: 54
Ingrese el numero 24: 7
Ingrese el numero 25: 9
Ingrese el numero 26: 54
Ingrese el numero 27: 28
Ingrese el numero 28: 10
Ingrese el numero 29: 43
Ingrese el numero 30: 124
Ingrese el numero 31: 45
Ingrese el numero 32: 694
Ingrese el numero 33: 58
Ingrese el numero 34: 3
Ingrese el numero 35: 6
Ingrese el numero 36: 7
Ingrese el numero 37: 95
Ingrese el numero 38: 54
Ingrese el numero 39: 9
Ingrese el numero 40: 5
Ingrese el numero 41: 493
Ingrese el numero 42: 56
Ingrese el numero 43: 220
Ingrese el numero 44: 59
Ingrese el numero 45: 32
Ingrese el numero 46: 59
Ingrese el numero 47: 960
Ingrese el numero 48: 54
Ingrese el numero 49: 90
Ingrese el numero 50: 64
La suma total es: 5000.0
La media es: 100.0
BUILD SUCCESSFUL (total time: 1 minute 39 seconds)
```

9- Calcular los múltiplos de 4 comprendidos entre N números enteros positivos.

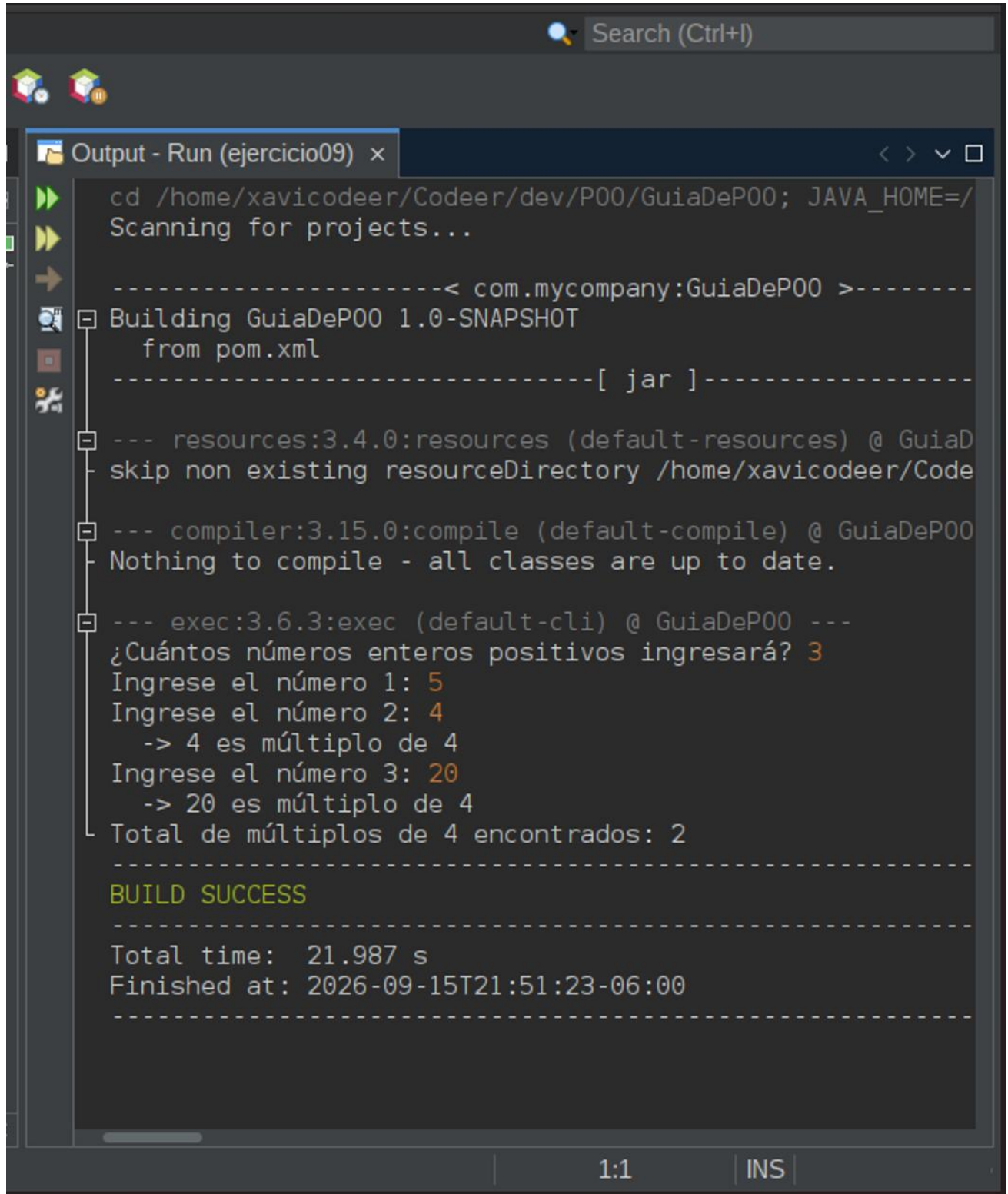
[Archivo de código \(NetBeans\)](#)



The screenshot shows the NetBeans IDE interface. The top menu bar includes Source, Refactor, Run, Debug, Profile, Team, Tools, Window, and Help. Below the menu bar is a toolbar with various icons. The main editor window displays the source code for 'ejercicio09.java'. The code is as follows:

```
1 package ejercicio09;
2
3 import java.util.Scanner;
4
5 public class ejercicio09 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int n, contador = 1, multiplos = 0;
10
11
12         do {
13             System.out.print("¿Cuántos números enteros positivos ingresará? ");
14             n = sc.nextInt();
15             if (n <= 0) {
16                 System.out.println("Error: N debe ser un entero positivo.");
17             }
18         } while (n <= 0);
19
20
21         while (contador <= n) {
22             System.out.print("Ingrese el número " + contador + ": ");
23             int num = sc.nextInt();
24
25             if (num > 0 && num % 4 == 0) {
26                 System.out.println(" -> " + num + " es múltiplo de 4");
27                 multiplos++;
28             }
29             contador++;
30         }
31
32         System.out.println("Total de múltiplos de 4 encontrados: " + multiplos);
33     }
34 }
```

Consola de salida (Ejercicio 9)



```
cd /home/xavicodeer/Codeer/dev/P00/GuiaDeP00; JAVA_HOME=/
Scanning for projects...

-----< com.mycompany:GuiaDeP00 >-----
Building GuiaDeP00 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----

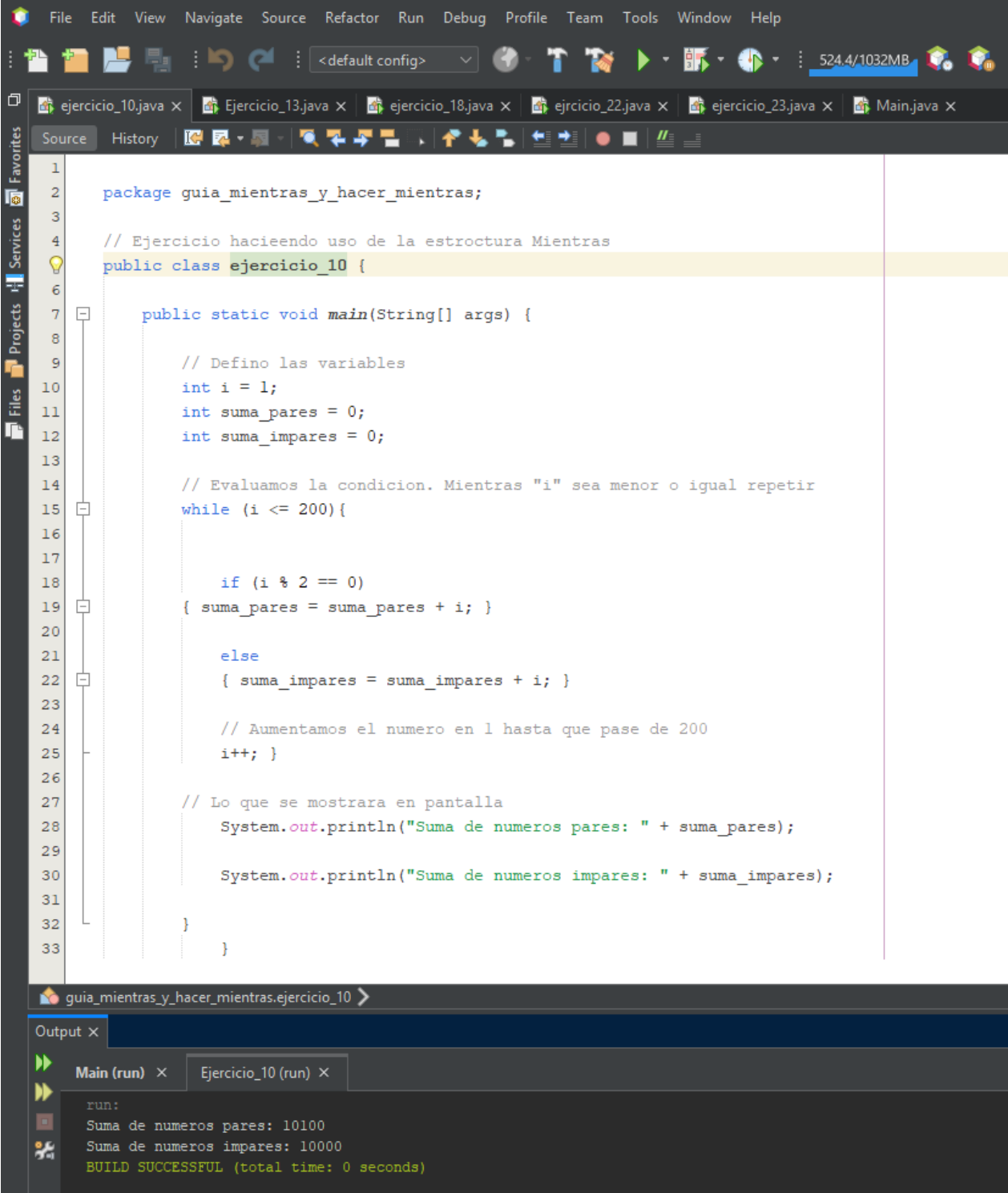
--- resources:3.4.0:resources (default-resources) @ GuiaD
- skip non existing resourceDirectory /home/xavicodeer/Code
-
--- compiler:3.15.0:compile (default-compile) @ GuiaDeP00
- Nothing to compile - all classes are up to date.
-
--- exec:3.6.3:exec (default-cli) @ GuiaDeP00 ---
¿Cuántos números enteros positivos ingresará? 3
Ingrese el número 1: 5
Ingrese el número 2: 4
  -> 4 es múltiplo de 4
Ingrese el número 3: 20
  -> 20 es múltiplo de 4
Total de múltiplos de 4 encontrados: 2

BUILD SUCCESS

Total time: 21.987 s
Finished at: 2026-09-15T21:51:23-06:00
```

10- Calcule e imprima la suma de los número pares e impares comprendidos entre 1 y 200

Archivo de código y consola de salida (ejercicio 10)



The screenshot shows an IDE with the following code in `ejercicio_10.java`:

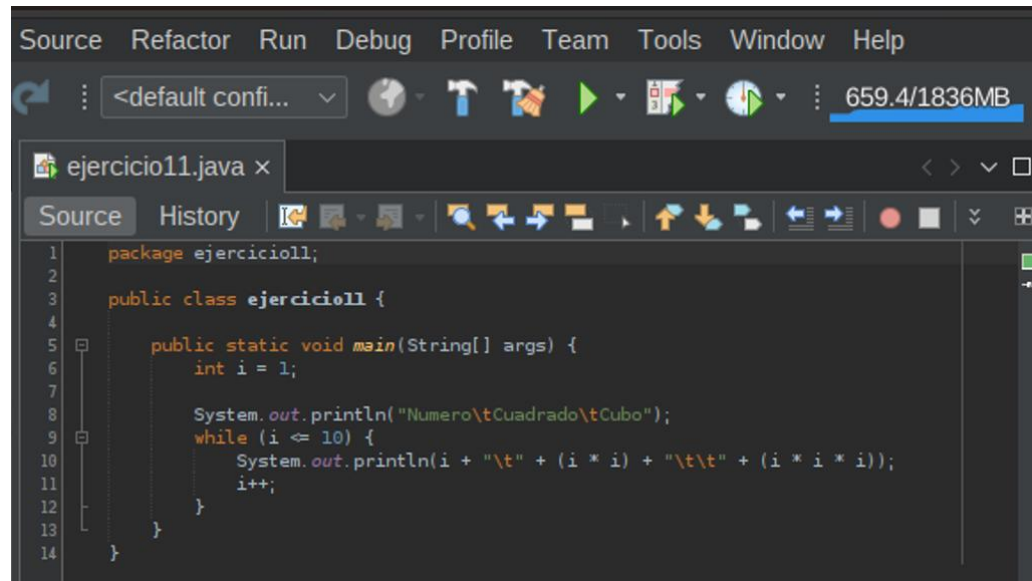
```
1 package guia_mientras_y_hacer_mientras;
2
3
4 // Ejercicio haciendo uso de la estroctura Mientras
5 public class ejercicio_10 {
6
7     public static void main(String[] args) {
8
9         // Defino las variables
10        int i = 1;
11        int suma_pares = 0;
12        int suma_impares = 0;
13
14        // Evaluamos la condicion. Mientras "i" sea menor o igual repetir
15        while (i <= 200) {
16
17
18            if (i % 2 == 0)
19            { suma_pares = suma_pares + i; }
20
21            else
22            { suma_impares = suma_impares + i; }
23
24            // Aumentamos el numero en 1 hasta que pase de 200
25            i++; }
26
27        // Lo que se mostrara en pantalla
28        System.out.println("Suma de numeros pares: " + suma_pares);
29
30        System.out.println("Suma de numeros impares: " + suma_impares);
31
32    }
33 }
```

The console output shows the results of the program execution:

```
run:
Suma de numeros pares: 10100
Suma de numeros impares: 10000
BUILD SUCCESSFUL (total time: 0 seconds)
```

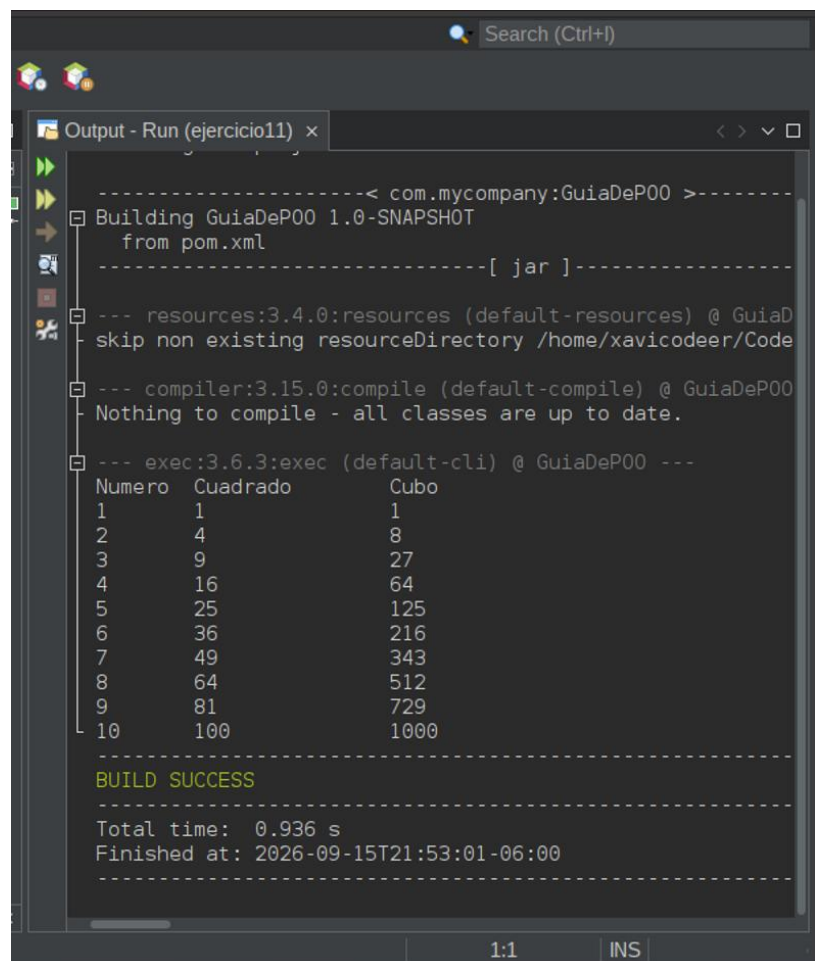
11- Imprimir los números del 1 al 10, sus cuadrados y sus cubos en forma de tablas.

Archivo de código (NetBeans)



```
1 package ejercicio11;
2
3 public class ejercicio11 {
4
5     public static void main(String[] args) {
6         int i = 1;
7
8         System.out.println("Numero\tCuadrado\tCubo");
9         while (i <= 10) {
10             System.out.println(i + "\t" + (i * i) + "\t\t" + (i * i * i));
11             i++;
12         }
13     }
14 }
```

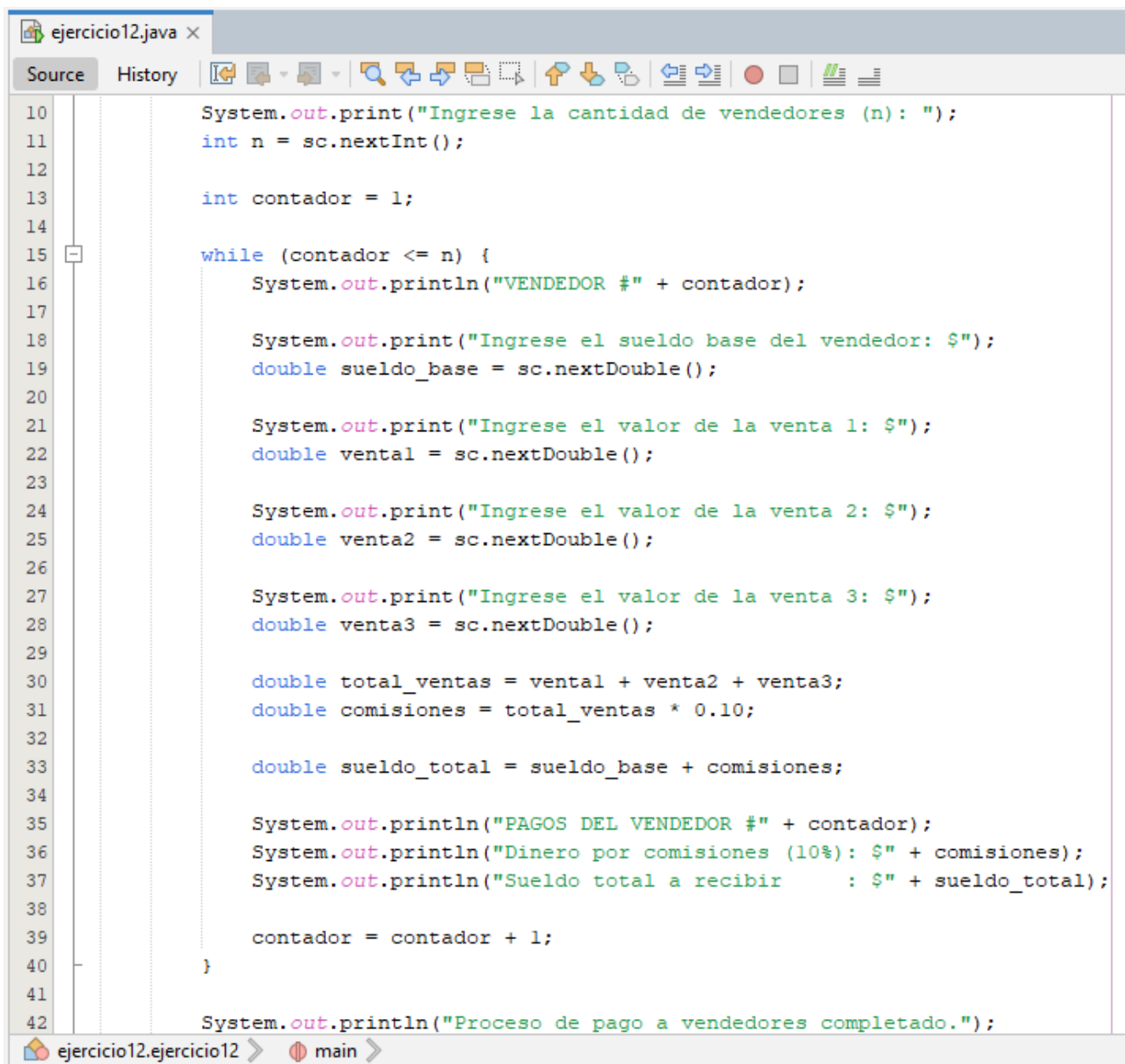
Consola de salida (ejercicio 11)



```
-----< com.mycompany:GuiaDeP00 >-----
Building GuiaDeP00 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----
--- resources:3.4.0:resources (default-resources) @ GuiaD
skip non existing resourceDirectory /home/xavicodeer/Code
--- compiler:3.15.0:compile (default-compile) @ GuiaDeP00
Nothing to compile - all classes are up to date.
--- exec:3.6.3:exec (default-cli) @ GuiaDeP00 ---
Numero  Cuadrado    Cubo
1         1          1
2         4          8
3         9         27
4        16         64
5        25        125
6        36        216
7        49        343
8        64        512
9        81        729
10       100       1000
BUILD SUCCESS
Total time: 0.936 s
Finished at: 2026-09-15T21:53:01-06:00
-----
```

12- Una compañía de seguros tiene contratados a n vendedores, cada uno de los cuales hace tres ventas a la semana. Su política de pago es que un vendedor recibe un sueldo base, más un 10% extra por comisiones de sus ventas. El gerente de la compañía desea saber cuánto dinero obtendrá en la semana cada vendedor por concepto de comisiones por las tres ventas realizadas, y cuanto tomando en cuenta su sueldo base y sus comisiones.

[Archivo de código \(NetBeans\)](#)

The image shows a screenshot of the NetBeans IDE with a file named 'ejercicio12.java' open. The code is written in Java and implements a loop to process payment for multiple salespeople. It prompts the user for the number of salespeople, then for each one, it asks for their base salary and three sales amounts. It then calculates the total sales, the 10% commission, and the total salary to be paid. The code is as follows:

```
10      System.out.print("Ingrese la cantidad de vendedores (n): ");
11      int n = sc.nextInt();
12
13      int contador = 1;
14
15      while (contador <= n) {
16          System.out.println("VENDEDOR #" + contador);
17
18          System.out.print("Ingrese el sueldo base del vendedor: $");
19          double sueldo_base = sc.nextDouble();
20
21          System.out.print("Ingrese el valor de la venta 1: $");
22          double vental = sc.nextDouble();
23
24          System.out.print("Ingrese el valor de la venta 2: $");
25          double venta2 = sc.nextDouble();
26
27          System.out.print("Ingrese el valor de la venta 3: $");
28          double venta3 = sc.nextDouble();
29
30          double total_ventas = vental + venta2 + venta3;
31          double comisiones = total_ventas * 0.10;
32
33          double sueldo_total = sueldo_base + comisiones;
34
35          System.out.println("PAGOS DEL VENDEDOR #" + contador);
36          System.out.println("Dinero por comisiones (10%): $" + comisiones);
37          System.out.println("Sueldo total a recibir      : $" + sueldo_total);
38
39          contador = contador + 1;
40      }
41
42      System.out.println("Proceso de pago a vendedores completado.");
```

Consola de salida (Ejercicio 12)

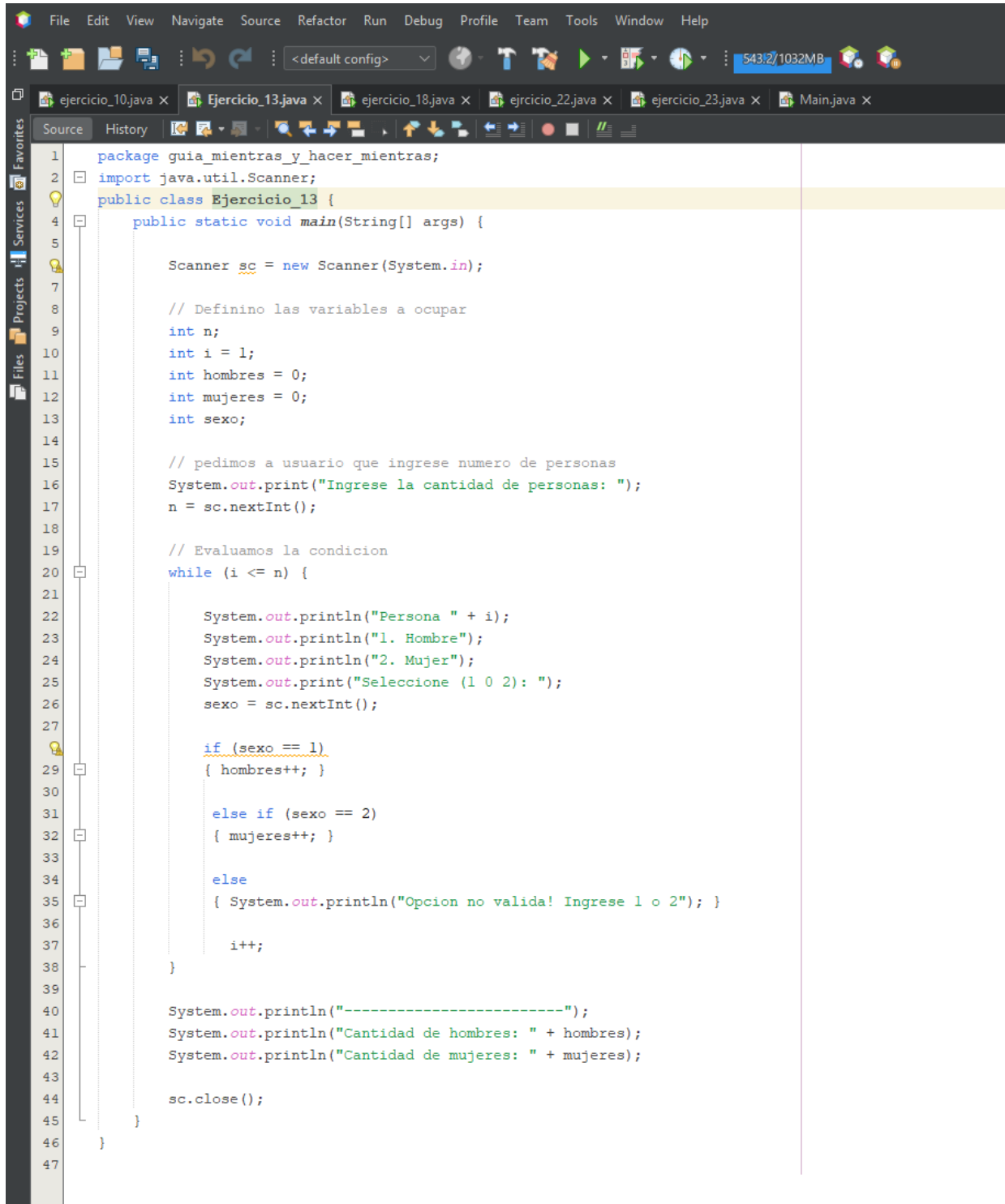
Output - EjerciciosGuiaPOO (run)



```
run:
Ingrese la cantidad de vendedores (n): 3
VENDEDOR #1
Ingrese el sueldo base del vendedor: $378
Ingrese el valor de la venta 1: $98
Ingrese el valor de la venta 2: $259
Ingrese el valor de la venta 3: $365
PAGOS DEL VENDEDOR #1
Dinero por comisiones (10%): $72.2
Sueldo total a recibir      : $450.2
VENDEDOR #2
Ingrese el sueldo base del vendedor: $426
Ingrese el valor de la venta 1: $167
Ingrese el valor de la venta 2: $84
Ingrese el valor de la venta 3: $478
PAGOS DEL VENDEDOR #2
Dinero por comisiones (10%): $72.9
Sueldo total a recibir      : $498.9
VENDEDOR #3
Ingrese el sueldo base del vendedor: $493
Ingrese el valor de la venta 1: $289
Ingrese el valor de la venta 2: $125
Ingrese el valor de la venta 3: $36
PAGOS DEL VENDEDOR #3
Dinero por comisiones (10%): $45.0
Sueldo total a recibir      : $538.0
Proceso de pago a vendedores completado.
BUILD SUCCESSFUL (total time: 1 minute 55 seconds)
```

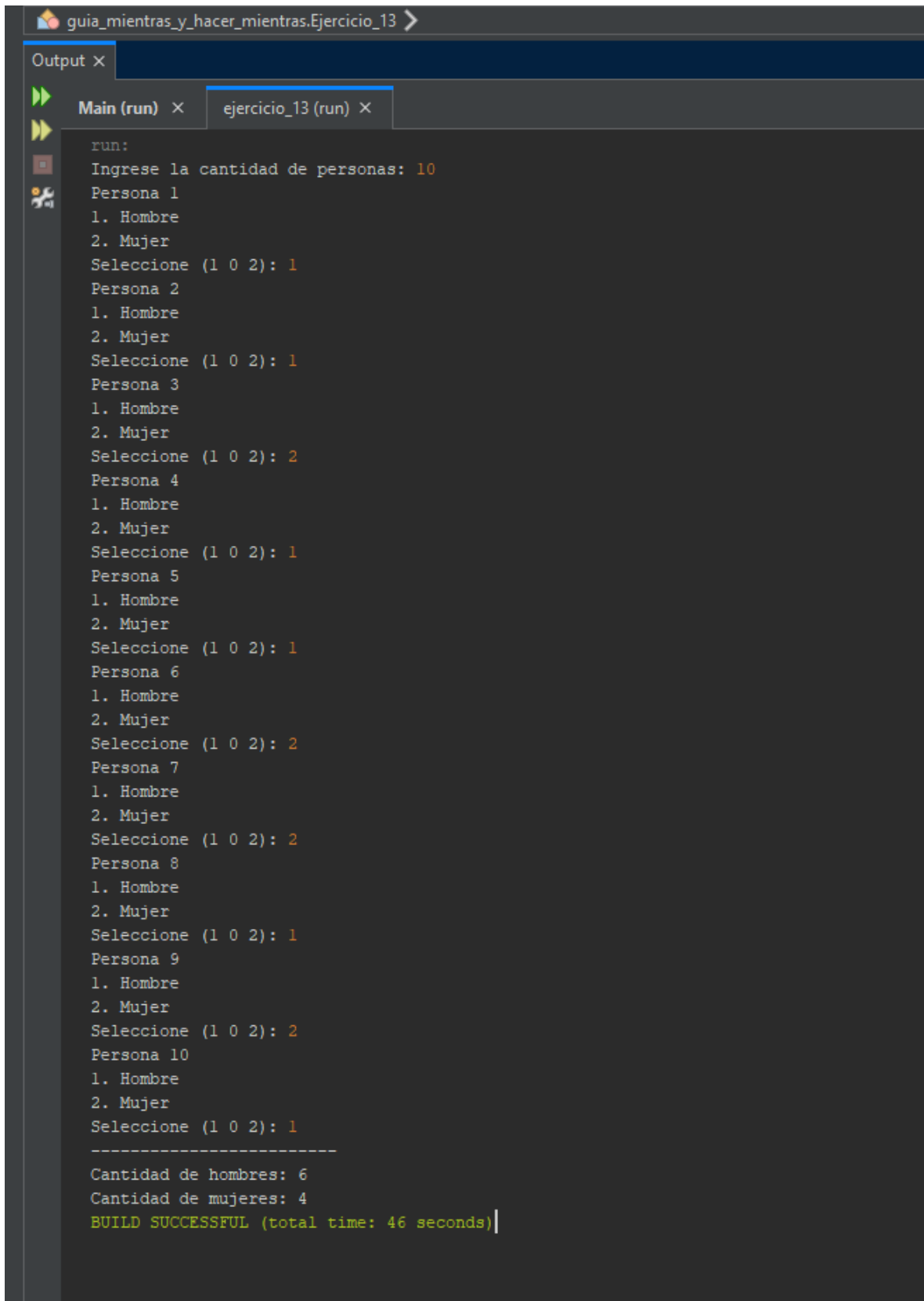
13- Determinar cuántos hombres y cuántas mujeres se encuentran en un grupo de n personas, suponiendo que los datos son extraídos alumno por alumno.

Archivo de código (NetBeans)



```
1 package guia_mientras_y_hacer_mientras;
2 import java.util.Scanner;
3 public class Ejercicio_13 {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7
8         // Definino las variables a ocupar
9         int n;
10        int i = 1;
11        int hombres = 0;
12        int mujeres = 0;
13        int sexo;
14
15        // pedimos a usuario que ingrese numero de personas
16        System.out.print("Ingrese la cantidad de personas: ");
17        n = sc.nextInt();
18
19        // Evaluamos la condicion
20        while (i <= n) {
21
22            System.out.println("Persona " + i);
23            System.out.println("1. Hombre");
24            System.out.println("2. Mujer");
25            System.out.print("Seleccione (1 0 2): ");
26            sexo = sc.nextInt();
27
28            if (sexo == 1)
29            { hombres++; }
30
31            else if (sexo == 2)
32            { mujeres++; }
33
34            else
35            { System.out.println("Opcion no valida! Ingrese 1 o 2"); }
36
37            i++;
38        }
39
40        System.out.println("-----");
41        System.out.println("Cantidad de hombres: " + hombres);
42        System.out.println("Cantidad de mujeres: " + mujeres);
43
44        sc.close();
45    }
46 }
47 }
```

Consola de salida (Ejercicio 13)



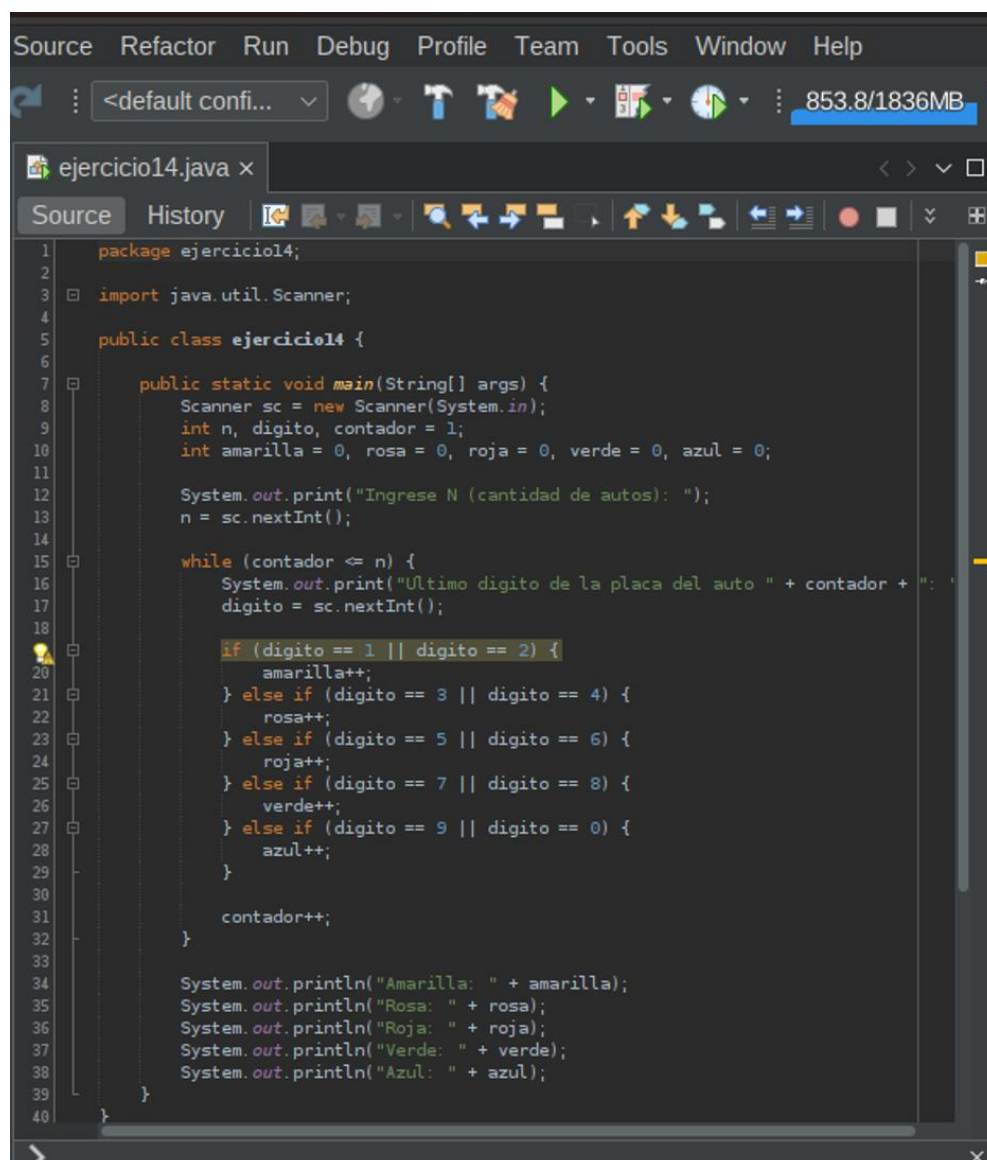
```
run:
Ingrese la cantidad de personas: 10
Persona 1
1. Hombre
2. Mujer
Seleccione (1 0 2): 1
Persona 2
1. Hombre
2. Mujer
Seleccione (1 0 2): 1
Persona 3
1. Hombre
2. Mujer
Seleccione (1 0 2): 2
Persona 4
1. Hombre
2. Mujer
Seleccione (1 0 2): 1
Persona 5
1. Hombre
2. Mujer
Seleccione (1 0 2): 1
Persona 6
1. Hombre
2. Mujer
Seleccione (1 0 2): 2
Persona 7
1. Hombre
2. Mujer
Seleccione (1 0 2): 2
Persona 8
1. Hombre
2. Mujer
Seleccione (1 0 2): 1
Persona 9
1. Hombre
2. Mujer
Seleccione (1 0 2): 2
Persona 10
1. Hombre
2. Mujer
Seleccione (1 0 2): 1
-----
Cantidad de hombres: 6
Cantidad de mujeres: 4
BUILD SUCCESSFUL (total time: 46 seconds)|
```

14- El departamento de Seguridad Pública y Tránsito, desea saber: de N autos que entran a la ciudad de El Salvador cuántos entran con calcomanía de cada color. Conociendo el último dígito de la placa de cada automóvil, se puede determinar el color de la calcomanía utilizando la siguiente relación:

Dígito		
1	ó	2
3	ó	4
5	ó	6
7	ó	8
9	ó	0

Color
Amarilla
Rosa
Roja
Verde
Azul

Archivo de código (NetBeans)

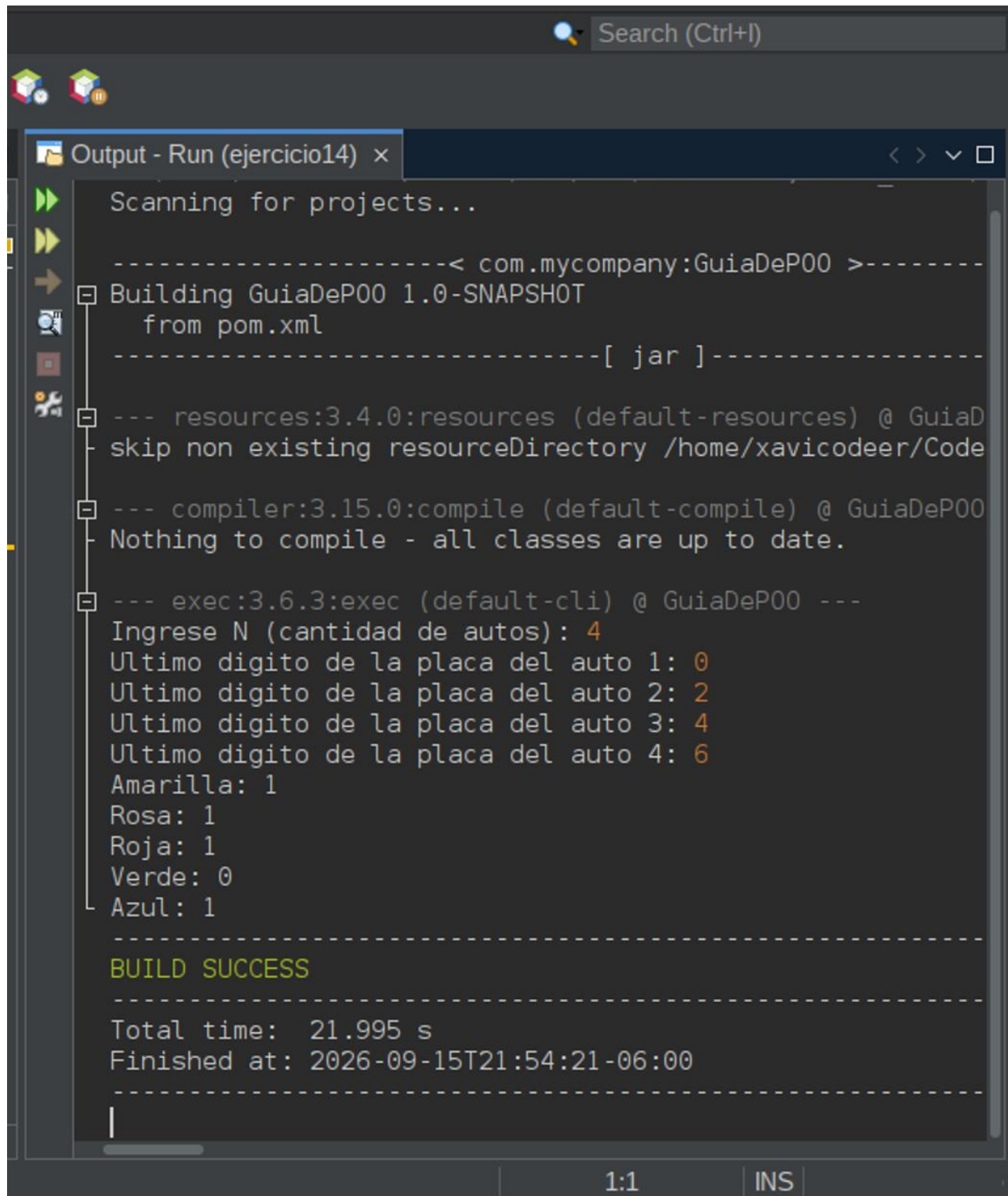


```

1 package ejercicio14;
2
3 import java.util.Scanner;
4
5 public class ejercicio14 {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         int n, digito, contador = 1;
10        int amarilla = 0, rosa = 0, roja = 0, verde = 0, azul = 0;
11
12        System.out.print("Ingrese N (cantidad de autos): ");
13        n = sc.nextInt();
14
15        while (contador <= n) {
16            System.out.print("Ultimo digito de la placa del auto " + contador + ": ");
17            digito = sc.nextInt();
18
19            if (digito == 1 || digito == 2) {
20                amarilla++;
21            } else if (digito == 3 || digito == 4) {
22                rosa++;
23            } else if (digito == 5 || digito == 6) {
24                roja++;
25            } else if (digito == 7 || digito == 8) {
26                verde++;
27            } else if (digito == 9 || digito == 0) {
28                azul++;
29            }
30
31            contador++;
32        }
33
34        System.out.println("Amarilla: " + amarilla);
35        System.out.println("Rosa: " + rosa);
36        System.out.println("Roja: " + roja);
37        System.out.println("Verde: " + verde);
38        System.out.println("Azul: " + azul);
39    }
40 }

```

Consola de salida (Ejercicio 14)



```
Search (Ctrl+I)

Output - Run (ejercicio14) x
Scanning for projects...
-----< com.mycompany:GuiaDeP00 >-----
Building GuiaDeP00 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----
--- resources:3.4.0:resources (default-resources) @ GuiaDeP00 ---
skip non existing resourceDirectory /home/xavicodeer/Code
--- compiler:3.15.0:compile (default-compile) @ GuiaDeP00 ---
Nothing to compile - all classes are up to date.
--- exec:3.6.3:exec (default-cli) @ GuiaDeP00 ---
Ingrese N (cantidad de autos): 4
Ultimo digito de la placa del auto 1: 0
Ultimo digito de la placa del auto 2: 2
Ultimo digito de la placa del auto 3: 4
Ultimo digito de la placa del auto 4: 6
Amarilla: 1
Rosa: 1
Roja: 1
Verde: 0
Azul: 1
-----
BUILD SUCCESS
-----
Total time: 21.995 s
Finished at: 2026-09-15T21:54:21-06:00
-----
|
```

15- Una persona debe realizar un muestreo de N personas para determinar el promedio de peso de los niños, jóvenes, adultos y ancianos que existen en su zona habitacional. Para ello, conforme va encontrando a las personas, introduce los datos a su computadora, la cual, mediante un programa, las clasifica y despliega los cuatro promedios que la persona requiere.

Categorías	Edad		
Niños	0	-	12
Jóvenes	13	-	29
Adultos	30	-	59
Ancianos	60	-	Adelante

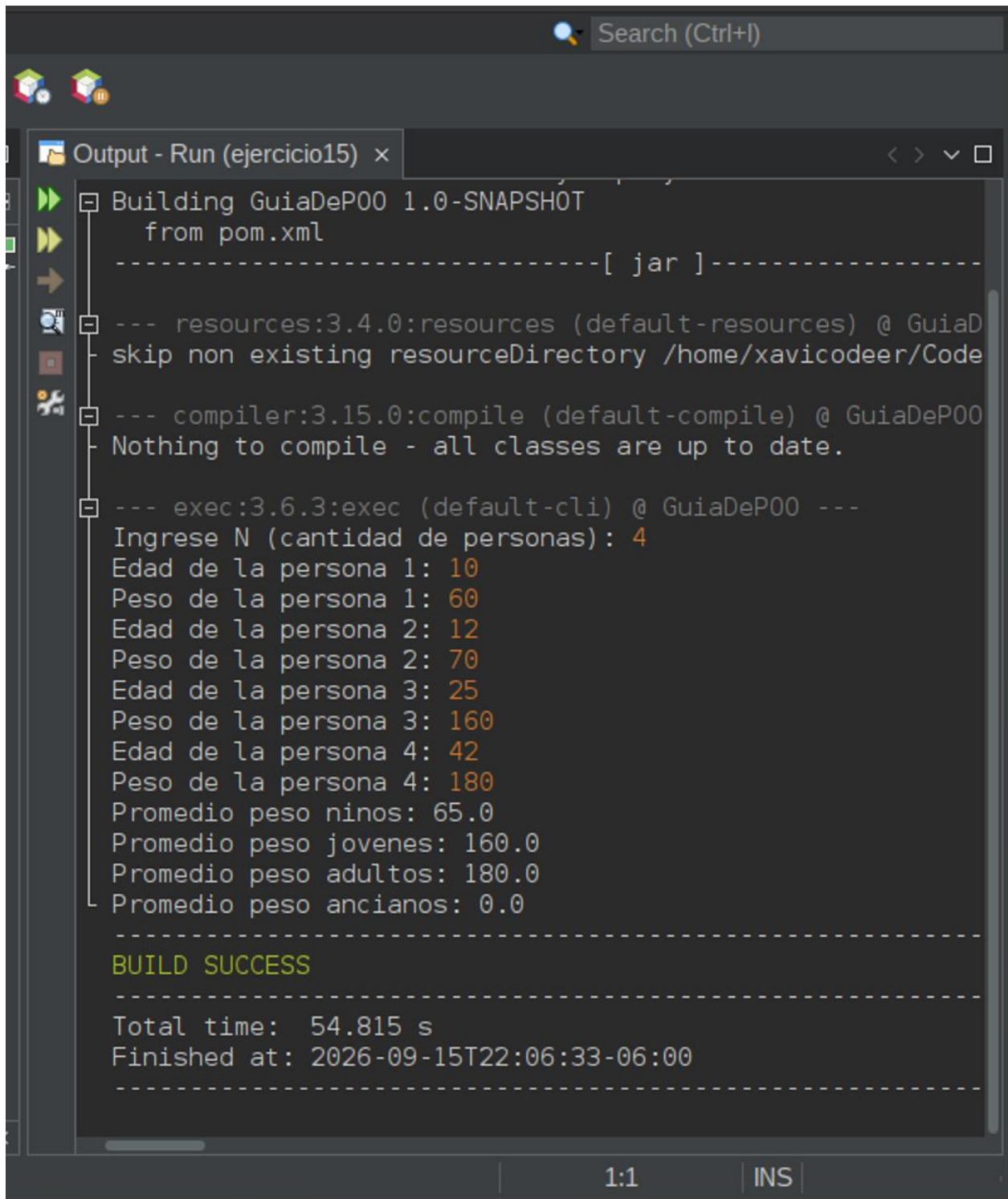
[Archivo de código \(NetBeans\)](#)

```

1  package ejercicio15;
2
3  import java.util.Scanner;
4
5  public class ejercicio15 {
6
7      public static void main(String[] args) {
8          Scanner sc = new Scanner(System.in);
9          int n, contador = 1, edad;
10         double peso;
11
12         double sumaNinos = 0, sumaJovenes = 0, sumaAdultos = 0, sumaAncianos = 0;
13         int cantNinos = 0, cantJovenes = 0, cantAdultos = 0, cantAncianos = 0;
14
15         System.out.print("Ingrese N (cantidad de personas): ");
16         n = sc.nextInt();
17
18         while (contador <= n) {
19             System.out.print("Edad de la persona " + contador + ": ");
20             edad = sc.nextInt();
21             System.out.print("Peso de la persona " + contador + ": ");
22             peso = sc.nextDouble();
23
24             if (edad <= 12) {
25                 sumaNinos += peso;
26                 cantNinos++;
27             } else if (edad <= 29) {
28                 sumaJovenes += peso;
29                 cantJovenes++;
30             } else if (edad <= 59) {
31                 sumaAdultos += peso;
32                 cantAdultos++;
33             } else {
34                 sumaAncianos += peso;
35                 cantAncianos++;
36             }
37
38             contador++;
39         }
40     }

```

Consola de salida (Ejercicio 15)



```
Search (Ctrl+I)

Output - Run (ejercicio15) x
Building GuiaDeP00 1.0-SNAPSHOT
  from pom.xml
-----[ jar ]-----
--- resources:3.4.0:resources (default-resources) @ GuiaD
skip non existing resourceDirectory /home/xavicodeer/Code
--- compiler:3.15.0:compile (default-compile) @ GuiaDeP00
Nothing to compile - all classes are up to date.
--- exec:3.6.3:exec (default-cli) @ GuiaDeP00 ---
Ingrese N (cantidad de personas): 4
Edad de la persona 1: 10
Peso de la persona 1: 60
Edad de la persona 2: 12
Peso de la persona 2: 70
Edad de la persona 3: 25
Peso de la persona 3: 160
Edad de la persona 4: 42
Peso de la persona 4: 180
Promedio peso ninos: 65.0
Promedio peso jovenes: 160.0
Promedio peso adultos: 180.0
Promedio peso ancianos: 0.0
-----
BUILD SUCCESS
-----
Total time: 54.815 s
Finished at: 2026-09-15T22:06:33-06:00
-----

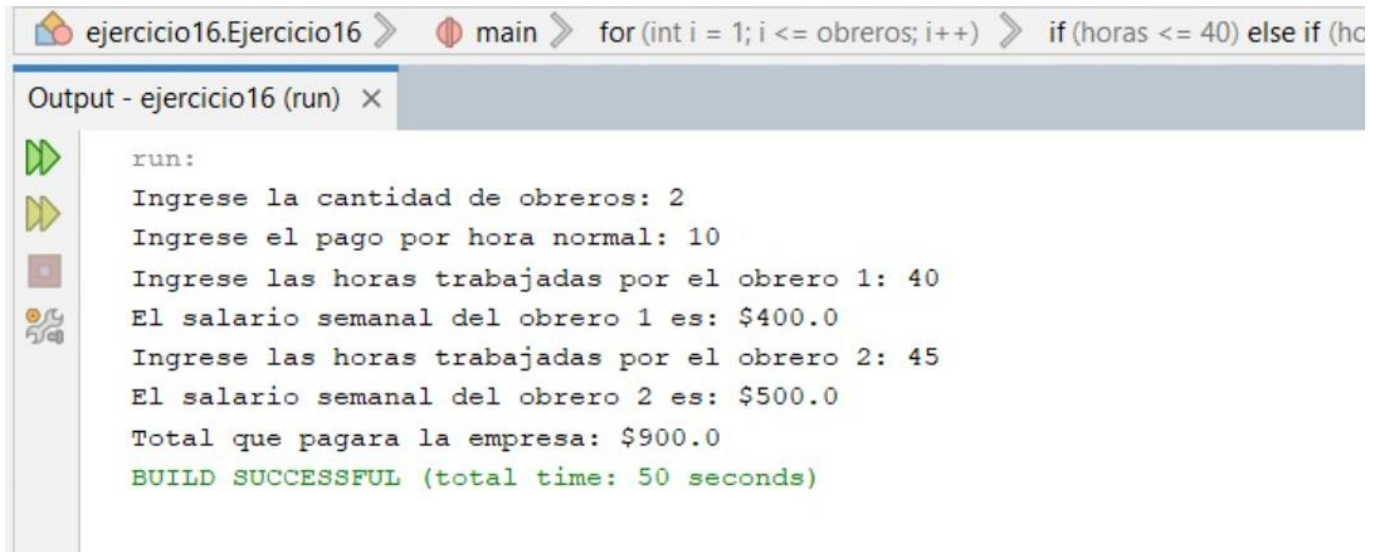
1:1 | INS |
```

16- Determine la cantidad semana de dinero que recibirá cada uno de los N obreros de la empresa. Se sabe que cuando las horas que trabajó un obrero exceden de 40 horas, el resto se convierte en horas extras que se pagan el doble de una hora normal, cuando exceden de 8; cuando las horas extras exceden de 8 se pagan las primeras 8 al doble de lo que paga una hora normal y el resto al triple. Además, la Empresa desea saber el total de dinero que pagara por concepto de pagos por salarios.

Archivo de código (NetBeans)

```
Ejercicio7.java x Ejercicio16.java x
Source History
1 package ejercicio16;
2
3 import java.util.Scanner;
4
5 public class Ejercicio16 {
6
7     public static void main(String[] args) {
8
9         Scanner entrada = new Scanner(System.in);
10
11         System.out.print("Ingrese la cantidad de obreros: ");
12         int obreros = entrada.nextInt();
13
14         System.out.print("Ingrese el pago por hora normal: ");
15         double pagoHora = entrada.nextDouble();
16
17         double totalEmpresa = 0;
18
19         for (int i = 1; i <= obreros; i++) {
20
21             System.out.print("Ingrese las horas trabajadas por el obrero " + i + ": ");
22             int horas = entrada.nextInt();
23
24             double salario;
25
26             if (horas <= 40) {
27                 salario = horas * pagoHora;
28             } else if (horas <= 48) {
29                 int horasExtra = horas - 40;
30                 salario = (40 * pagoHora) + (horasExtra * pagoHora * 2);
31             } else {
32                 int horasExtra = horas - 40;
33                 int primerasOcho = 8;
34                 int restoExtra = horasExtra - 8;
35
36                 salario = (40 * pagoHora)
37                     + (primerasOcho * pagoHora * 2)
38                     + (restoExtra * pagoHora * 3);
39             }
40
41             System.out.println("El salario semanal del obrero " + i + " es: $" + salario);
42
43             totalEmpresa = totalEmpresa + salario;
44         }
45
46         System.out.println("Total que pagara la empresa: $" + totalEmpresa);
47     }
48 }
```

Consola de salida (Ejercicio 16)



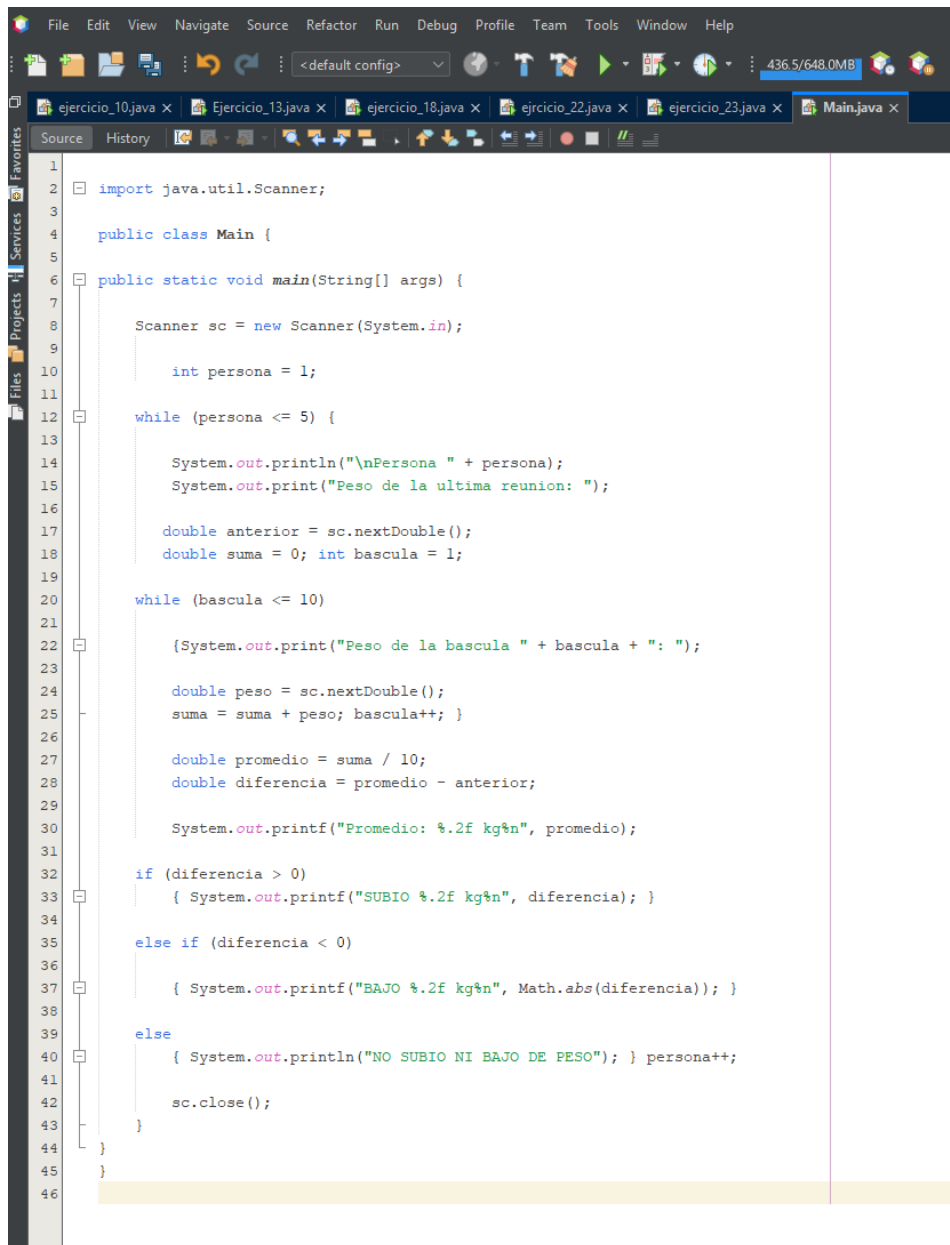
The screenshot shows a C++ IDE with a console window titled "Output - ejercicio16 (run)". The console displays the following text:

```
run:
Ingrese la cantidad de obreros: 2
Ingrese el pago por hora normal: 10
Ingrese las horas trabajadas por el obrero 1: 40
El salario semanal del obrero 1 es: $400.0
Ingrese las horas trabajadas por el obrero 2: 45
El salario semanal del obrero 2 es: $500.0
Total que pagara la empresa: $900.0
BUILD SUCCESSFUL (total time: 50 seconds)
```

The IDE interface includes a breadcrumb trail at the top: "ejercicio16.Ejercicio16 > main > for (int i = 1; i <= obreros; i++) > if (horas <= 40) else if (hc". On the left side of the console, there are icons for running (green play button), stepping through (yellow play button), and debugging (red stop button).

17- Cinco miembros de un club contra la obesidad desean saber cuánto han bajado o subido de peso desde la última vez que se reunieron. Para esto se debe realizar un ritual de pesaje en donde cada uno se pesa en diez básculas distintas para así tener el promedio exacto de su peso. Si existe diferencia positiva entre este promedio de peso y el peso de la última vez que se reunieron, significa que subieron de peso. Pero si la diferencia es negativa, significa que bajaron. Lo que el problema requiere es por cada persona se imprima un letrero que diga: “SUBIO” o “BAJO” y la cantidad de kilos que subió o bajo de peso.

Archivo de código (NetBeans)



```
1  import java.util.Scanner;
2
3
4  public class Main {
5
6  public static void main(String[] args) {
7
8      Scanner sc = new Scanner(System.in);
9
10     int persona = 1;
11
12     while (persona <= 5) {
13
14         System.out.println("\nPersona " + persona);
15         System.out.print("Peso de la ultima reunion: ");
16
17         double anterior = sc.nextDouble();
18         double suma = 0; int bascula = 1;
19
20         while (bascula <= 10)
21
22             {System.out.print("Peso de la bascula " + bascula + ": ");
23
24             double peso = sc.nextDouble();
25             suma = suma + peso; bascula++; }
26
27         double promedio = suma / 10;
28         double diferencia = promedio - anterior;
29
30         System.out.printf("Promedio: %.2f kg%n", promedio);
31
32         if (diferencia > 0)
33             { System.out.printf("SUBIO %.2f kg%n", diferencia); }
34
35         else if (diferencia < 0)
36
37             { System.out.printf("BAJO %.2f kg%n", Math.abs(diferencia)); }
38
39         else
40             { System.out.println("NO SUBIO NI BAJO DE PESO"); } persona++;
41
42         sc.close();
43     }
44 }
45
46
```

Consola de salida (Ejercicio 17)

```
Output - Main (run) ×
Output - Main (run)

Persona 1
Peso de la ultima reunion: 95
Peso de la bascula 1: 91
Peso de la bascula 2: 90
Peso de la bascula 3: 90
Peso de la bascula 4: 89
Peso de la bascula 5: 90
Peso de la bascula 6: 91
Peso de la bascula 7: 92
Peso de la bascula 8: 90
Peso de la bascula 9: 91
Peso de la bascula 10: 89
Promedio: 90.30 kg
BAJO 4.70 kg

Persona 2
Peso de la ultima reunion: 86.2
Peso de la bascula 1: 87
Peso de la bascula 2: 87.5
Peso de la bascula 3: 85.9
Peso de la bascula 4: 88.2
Peso de la bascula 5: 88.7
Peso de la bascula 6: 89
Peso de la bascula 7: 87.3
Peso de la bascula 8: 88
Peso de la bascula 9: 87.6
Peso de la bascula 10: 86.8
Promedio: 87.60 kg
SUBIO 1.40 kg

Persona 3
Peso de la ultima reunion: 97.5
Peso de la bascula 1: 93
Peso de la bascula 2: 94
Peso de la bascula 3: 93.5
Peso de la bascula 4: 94
Peso de la bascula 5: 95
Peso de la bascula 6: 93
Peso de la bascula 7: 93
Peso de la bascula 8: 94
Peso de la bascula 9: 92
Peso de la bascula 10: 92
Promedio: 93.35 kg
BAJO 4.15 kg
```

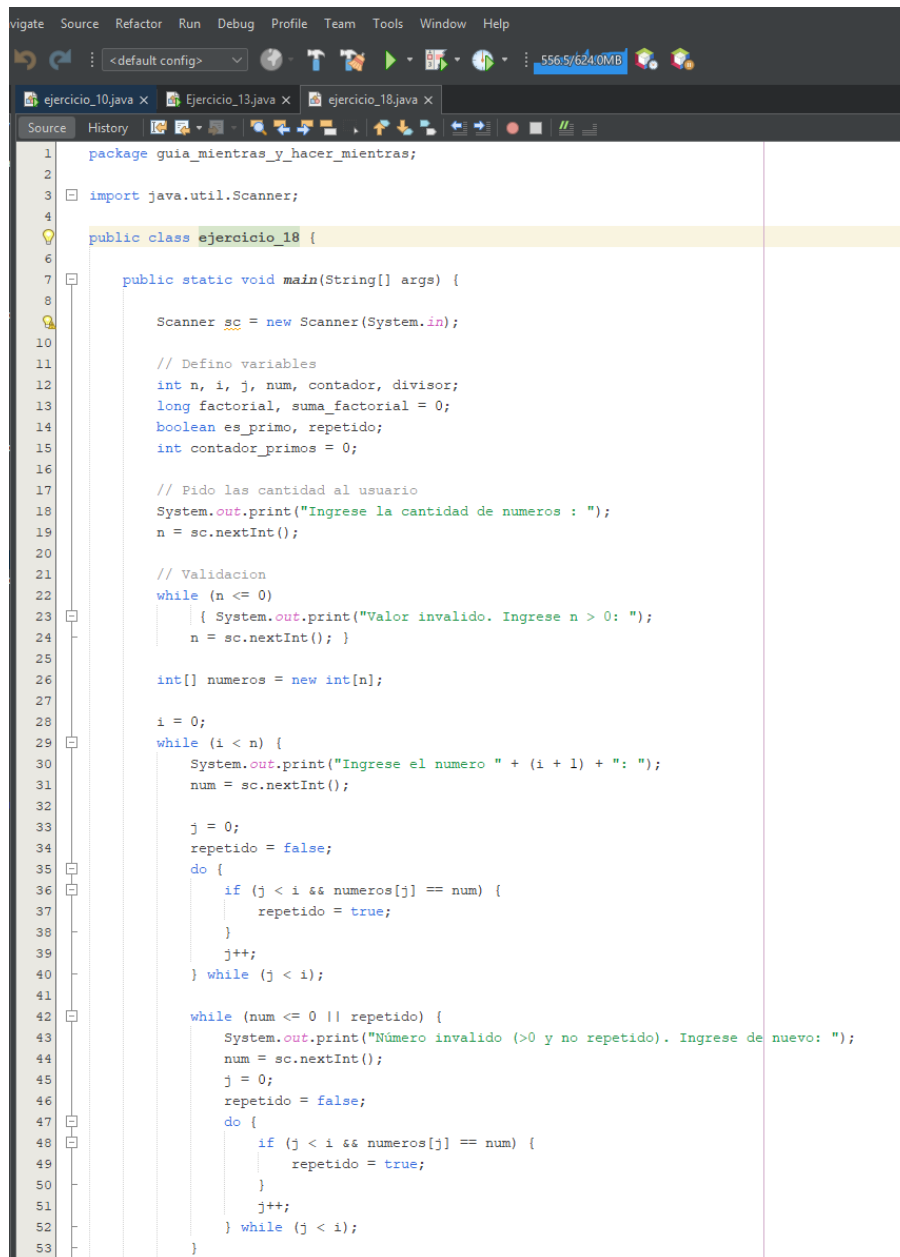
```
Persona 4
Peso de la ultima reunion: 105
Peso de la bascula 1: 92
Peso de la bascula 2: 93
Peso de la bascula 3: 90
Peso de la bascula 4: 93
Peso de la bascula 5: 92
Peso de la bascula 6: 94
Peso de la bascula 7: 91
Peso de la bascula 8: 90
Peso de la bascula 9: 92
Peso de la bascula 10: 93
Promedio: 92.00 kg
BAJO 13.00 kg

Persona 5
Peso de la ultima reunion: 89.7
Peso de la bascula 1: 108.9
Peso de la bascula 2: 110
Peso de la bascula 3: 109.3
Peso de la bascula 4: 108.1
Peso de la bascula 5: 110.9
Peso de la bascula 6: 112.4
Peso de la bascula 7: 111.1
Peso de la bascula 8: 110
Peso de la bascula 9: 110.6
Peso de la bascula 10: 111.7
Promedio: 110.30 kg
SUBIO 20.60 kg
BUILD SUCCESSFUL (total time: 18 minutes 28 seconds)
```

18- Desarrollar un pseudocódigo tal, que dado N números enteros como datos, el número que ingrese de N números deberá ser diferentes y mayores de cero:

- De los “n” números ingresados calcular e imprimir los números primos y diga cuántos primos son.
- De los “n” números ingresados calcule e imprima la factorial de cada uno de ellos y la suma del factorial.
- De los “n” números ingresados calcula e imprima, cuántos son pares e impares.

Archivo de código (NetBeans 2 capturas)



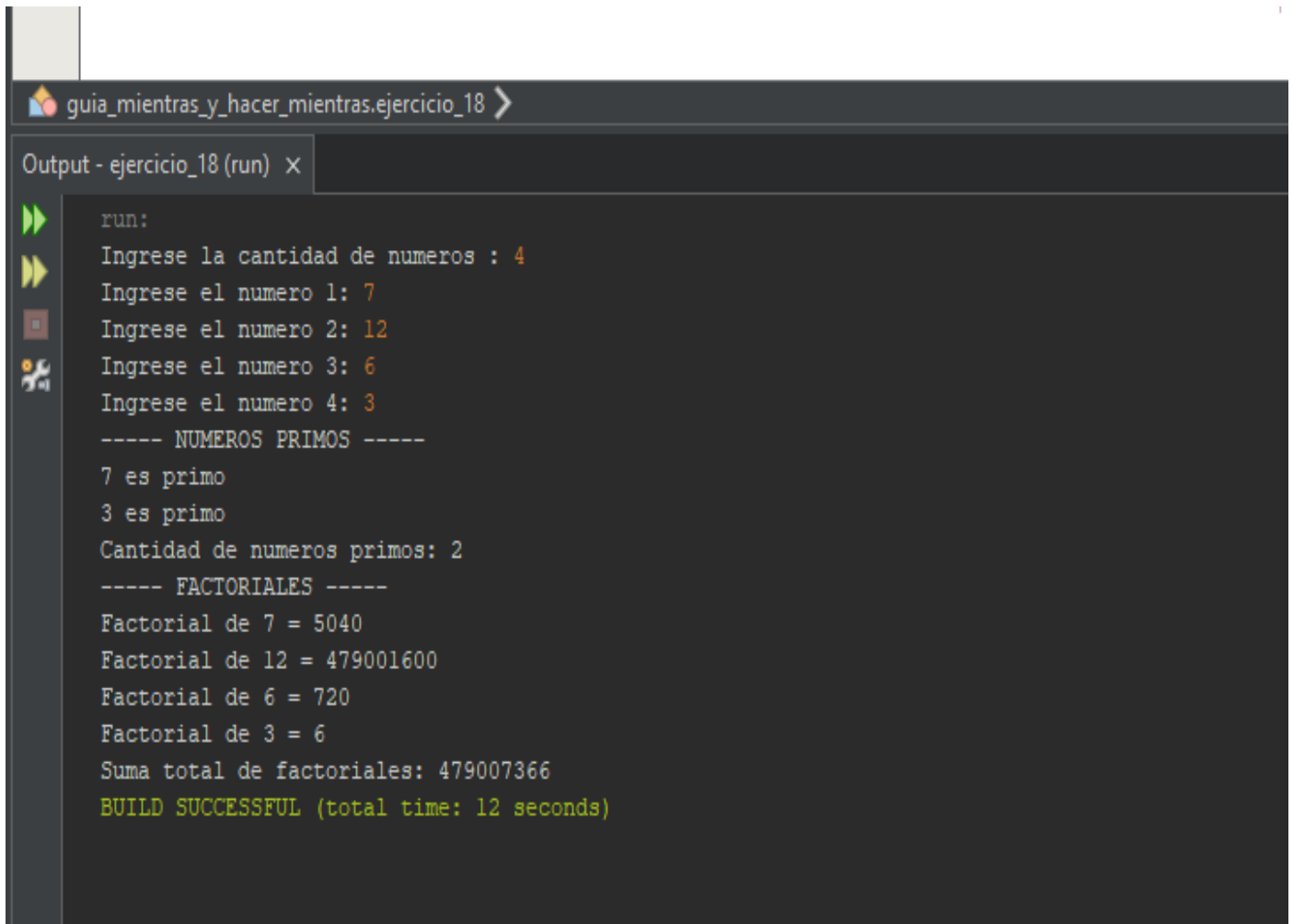
```
1 package guia_mientras_y_hacer_mientras;
2
3 import java.util.Scanner;
4
5 public class ejercicio_18 {
6
7     public static void main(String[] args) {
8
9         Scanner sc = new Scanner(System.in);
10
11         // Defino variables
12         int n, i, j, num, contador, divisor;
13         long factorial, suma_factorial = 0;
14         boolean es_primo, repetido;
15         int contador_primos = 0;
16
17         // Pido las cantidad al usuario
18         System.out.print("Ingrese la cantidad de numeros : ");
19         n = sc.nextInt();
20
21         // Validacion
22         while (n <= 0)
23             { System.out.print("Valor invalido. Ingrese n > 0: ");
24               n = sc.nextInt(); }
25
26         int[] numeros = new int[n];
27
28         i = 0;
29         while (i < n) {
30             System.out.print("Ingrese el numero " + (i + 1) + ": ");
31             num = sc.nextInt();
32
33             j = 0;
34             repetido = false;
35             do {
36                 if (j < i && numeros[j] == num) {
37                     repetido = true;
38                 }
39                 j++;
40             } while (j < i);
41
42             while (num <= 0 || repetido) {
43                 System.out.print("Número invalido (>0 y no repetido). Ingrese de nuevo: ");
44                 num = sc.nextInt();
45                 j = 0;
46                 repetido = false;
47                 do {
48                     if (j < i && numeros[j] == num) {
49                         repetido = true;
50                     }
51                     j++;
52                 } while (j < i);
53             }
54         }
55     }
```

```

54         numeros[i] = num;
55         i++;
56     }
57
58
59     // Cálculo de números primos
60     System.out.println("----- NUMEROS PRIMOS -----");
61     i = 0;
62     while (i < n) {
63         num = numeros[i];
64         es_primo = true;
65
66         if (num == 1) {
67             es_primo = false;
68         } else {
69             divisor = 2;
70             while (divisor <= num / 2 && es_primo) {
71                 if (num % divisor == 0) {
72                     es_primo = false;
73                 }
74                 divisor++;
75             }
76
77             if (es_primo) {
78                 System.out.println(num + " es primo");
79                 contador_primos++;
80             }
81             i++;
82         }
83     }
84     System.out.println("Cantidad de numeros primos: " + contador_primos);
85
86     // Cálculo de factoriales
87     System.out.println("----- FACTORIALES -----");
88     i = 0;
89     while (i < n) {
90         num = numeros[i];
91         factorial = 1;
92         contador = 1;
93         while (contador <= num) {
94             factorial = factorial * contador;
95             contador++;
96         }
97
98         System.out.println("Factorial de " + num + " = " + factorial);
99         suma_factorial += factorial;
100        i++;
101    }
102    System.out.println("Suma total de factoriales: " + suma_factorial);
103
104    sc.close();
105 }
106

```

Consola de salida (Ejercicio 18)

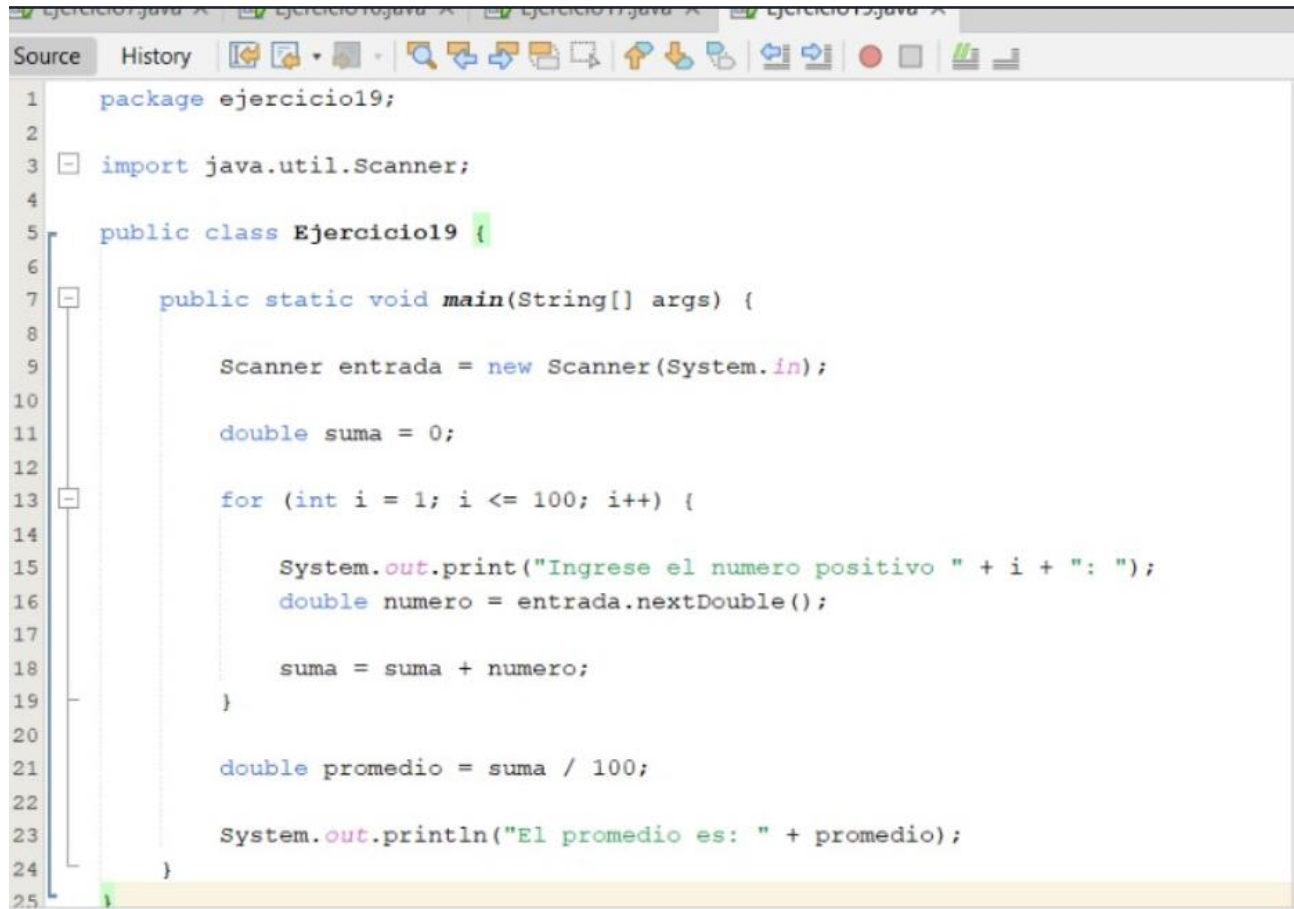


The screenshot shows a terminal window with a dark background. The title bar at the top reads "guia_mientras_y_hacer_mientras.ejercicio_18" followed by a right-pointing arrow. Below the title bar, there is a tab labeled "Output - ejercicio_18 (run) x". The main area of the terminal displays the following text:

```
run:
Ingrese la cantidad de numeros : 4
Ingrese el numero 1: 7
Ingrese el numero 2: 12
Ingrese el numero 3: 6
Ingrese el numero 4: 3
----- NUMEROS PRIMOS -----
7 es primo
3 es primo
Cantidad de numeros primos: 2
----- FACTORIALES -----
Factorial de 7 = 5040
Factorial de 12 = 479001600
Factorial de 6 = 720
Factorial de 3 = 6
Suma total de factoriales: 479007366
BUILD SUCCESSFUL (total time: 12 seconds)
```

19- Calcular e imprimir el promedio de 100 numero positivos.

[Archivo de código \(NetBeans\)](#)



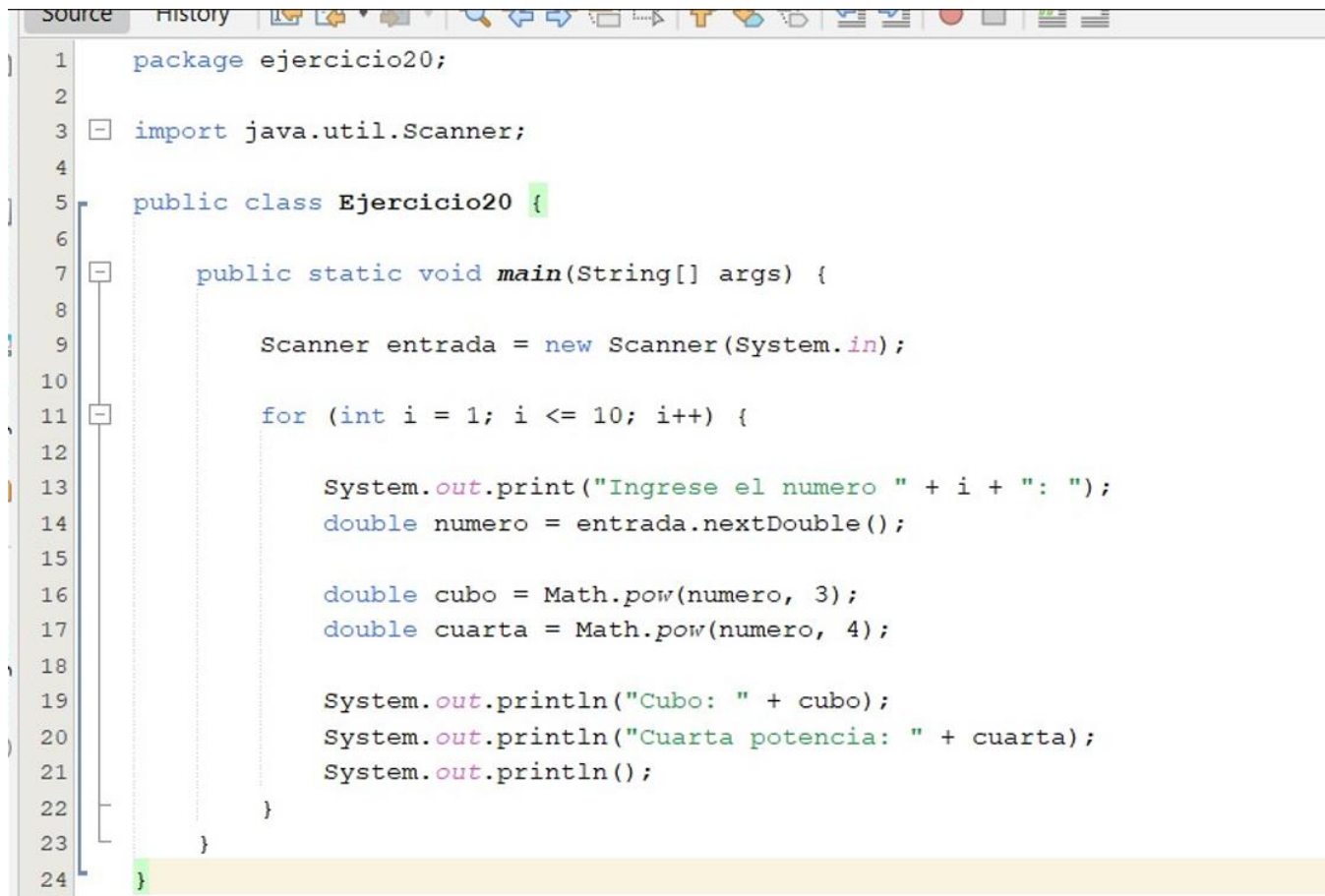
```
1 package ejercicio19;
2
3 import java.util.Scanner;
4
5 public class Ejercicio19 {
6
7     public static void main(String[] args) {
8
9         Scanner entrada = new Scanner(System.in);
10
11         double suma = 0;
12
13         for (int i = 1; i <= 100; i++) {
14
15             System.out.print("Ingrese el numero positivo " + i + ": ");
16             double numero = entrada.nextDouble();
17
18             suma = suma + numero;
19         }
20
21         double promedio = suma / 100;
22
23         System.out.println("El promedio es: " + promedio);
24     }
25 }
```

Consola de salida (Ejercicio 19)

```
Output - ejercicio19 (run) X
run:
Ingrese el numero positivo 1: 1
Ingrese el numero positivo 2: 2
Ingrese el numero positivo 3: 3
Ingrese el numero positivo 4: 4
Ingrese el numero positivo 5: 5
Ingrese el numero positivo 6: 6
Ingrese el numero positivo 7: 7
Ingrese el numero positivo 8: 8
Ingrese el numero positivo 9: 9
Ingrese el numero positivo 10: 10
Ingrese el numero positivo 11: 11
Ingrese el numero positivo 12: 12
Ingrese el numero positivo 13: 13
Ingrese el numero positivo 14: 14
Ingrese el numero positivo 15: 15
Ingrese el numero positivo 16: 16
Ingrese el numero positivo 17: 17
Ingrese el numero positivo 18: 18
Ingrese el numero positivo 19: 19
Ingrese el numero positivo 20: 20
Ingrese el numero positivo 21: 21
Ingrese el numero positivo 22: 22
Ingrese el numero positivo 23: 23
Ingrese el numero positivo 24: 24
Ingrese el numero positivo 25: 25
Ingrese el numero positivo 26: 26
Ingrese el numero positivo 27: 27
Ingrese el numero positivo 28: 28
Ingrese el numero positivo 29: 29
Ingrese el numero positivo 30: 30
Ingrese el numero positivo 31: 31
Ingrese el numero positivo 32: 32
Ingrese el numero positivo 33: 33
Ingrese el numero positivo 34: 34
Ingrese el numero positivo 35: 35
Ingrese el numero positivo 36: 36
Ingrese el numero positivo 37: 37
Ingrese el numero positivo 38: 38
Ingrese el numero positivo 39: 39
Ingrese el numero positivo 40: 40
Ingrese el numero positivo 41: 41
Ingrese el numero positivo 42: 42
Ingrese el numero positivo 43: 43
Ingrese el numero positivo 44: 44
Ingrese el numero positivo 45: 45
Ingrese el numero positivo 46: 46
Ingrese el numero positivo 47: 47
Ingrese el numero positivo 48: 48
Ingrese el numero positivo 49: 49
Ingrese el numero positivo 50: 50
Ingrese el numero positivo 51: 51
Ingrese el numero positivo 52: 52
Ingrese el numero positivo 53: 53
Ingrese el numero positivo 54: 54
Ingrese el numero positivo 55: 55
Ingrese el numero positivo 56: 56
Ingrese el numero positivo 65: 65
Ingrese el numero positivo 66: 66
Ingrese el numero positivo 67: 67
Ingrese el numero positivo 68: 68
Ingrese el numero positivo 69: 69
Ingrese el numero positivo 70: 70
Ingrese el numero positivo 71: 71
Ingrese el numero positivo 72: 72
Ingrese el numero positivo 73: 73
Ingrese el numero positivo 74: 74
Ingrese el numero positivo 75: 75
Ingrese el numero positivo 76: 76
Ingrese el numero positivo 77: 77
Ingrese el numero positivo 78: 78
Ingrese el numero positivo 79: 79
Ingrese el numero positivo 80: 80
Ingrese el numero positivo 81: 81
Ingrese el numero positivo 82: 82
Ingrese el numero positivo 83: 83
Ingrese el numero positivo 84: 84
Ingrese el numero positivo 85: 85
Ingrese el numero positivo 86: 86
Ingrese el numero positivo 87: 87
Ingrese el numero positivo 88: 88
Ingrese el numero positivo 89: 89
Ingrese el numero positivo 90: 90
Ingrese el numero positivo 91: 91
Ingrese el numero positivo 92: 92
Ingrese el numero positivo 93: 93
Ingrese el numero positivo 94: 94
Ingrese el numero positivo 95: 95
Ingrese el numero positivo 96: 96
Ingrese el numero positivo 97: 97
Ingrese el numero positivo 98: 98
Ingrese el numero positivo 99: 99
Ingrese el numero positivo 100: 100
El promedio es: 50.5
BUILD SUCCESSFUL (total time: 5 minutes 49 seconds)
```

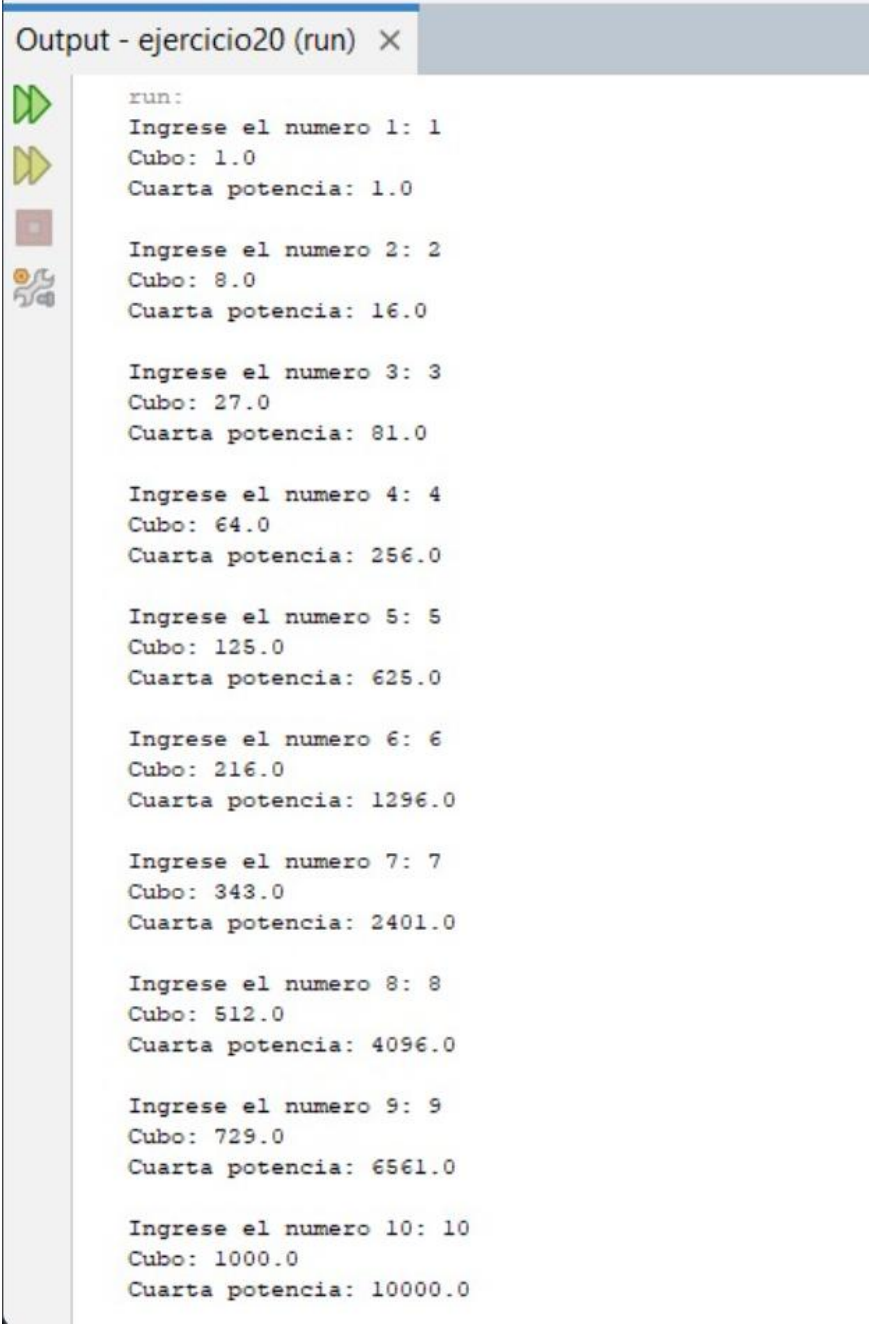
20- Leer 10 números y obtener su cubo y su cuarta.

[Archivo de código \(NetBeans\)](#)



```
1 package ejercicio20;
2
3 import java.util.Scanner;
4
5 public class Ejercicio20 {
6
7     public static void main(String[] args) {
8
9         Scanner entrada = new Scanner(System.in);
10
11         for (int i = 1; i <= 10; i++) {
12
13             System.out.print("Ingrese el numero " + i + ": ");
14             double numero = entrada.nextDouble();
15
16             double cubo = Math.pow(numero, 3);
17             double cuarta = Math.pow(numero, 4);
18
19             System.out.println("Cubo: " + cubo);
20             System.out.println("Cuarta potencia: " + cuarta);
21             System.out.println();
22         }
23     }
24 }
```

Consola de salida (Ejercicio 19)



```
Output - ejercicio20 (run) X
run:
Ingrese el numero 1: 1
Cubo: 1.0
Cuarta potencia: 1.0

Ingrese el numero 2: 2
Cubo: 8.0
Cuarta potencia: 16.0

Ingrese el numero 3: 3
Cubo: 27.0
Cuarta potencia: 81.0

Ingrese el numero 4: 4
Cubo: 64.0
Cuarta potencia: 256.0

Ingrese el numero 5: 5
Cubo: 125.0
Cuarta potencia: 625.0

Ingrese el numero 6: 6
Cubo: 216.0
Cuarta potencia: 1296.0

Ingrese el numero 7: 7
Cubo: 343.0
Cuarta potencia: 2401.0

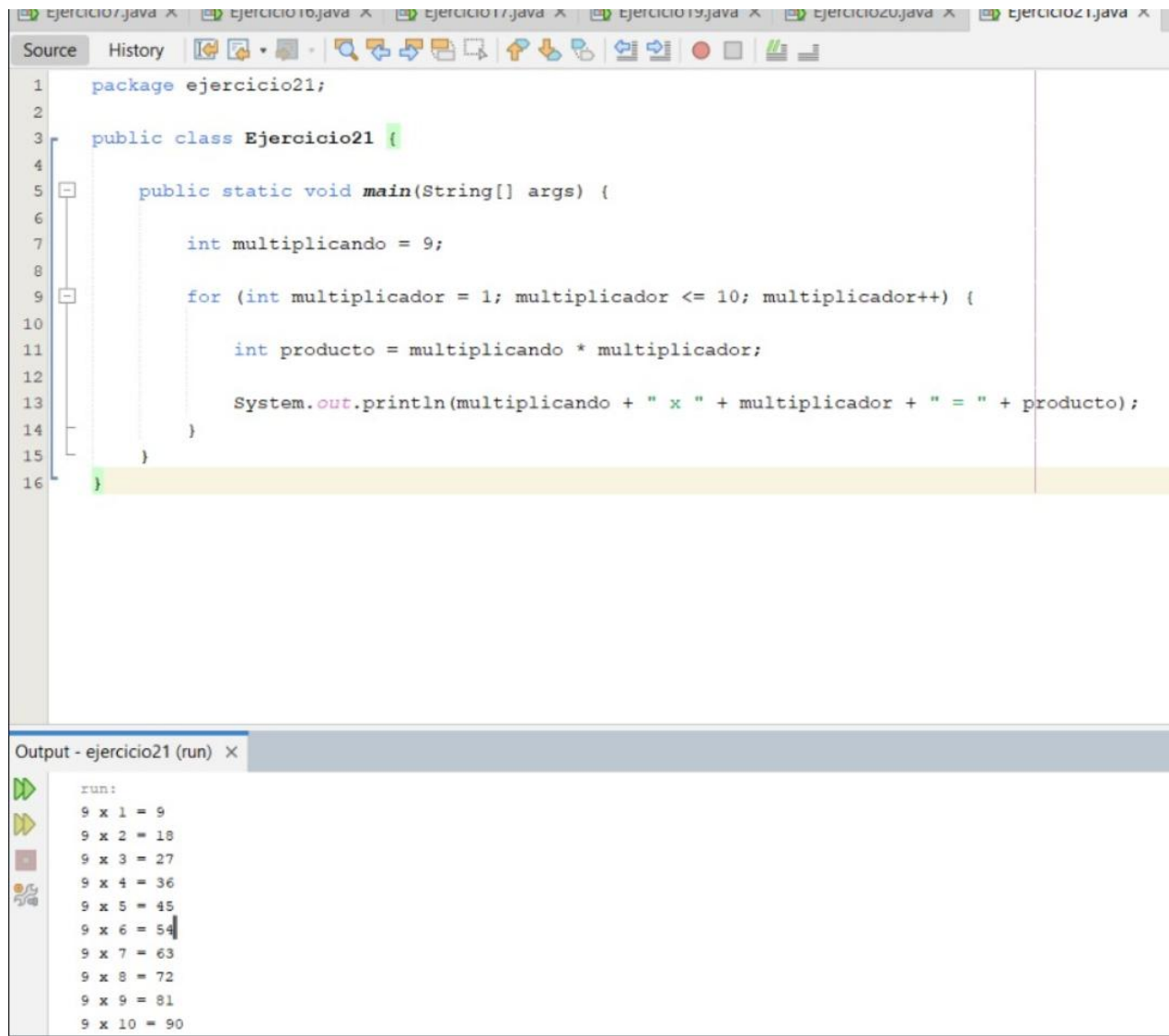
Ingrese el numero 8: 8
Cubo: 512.0
Cuarta potencia: 4096.0

Ingrese el numero 9: 9
Cubo: 729.0
Cuarta potencia: 6561.0

Ingrese el numero 10: 10
Cubo: 1000.0
Cuarta potencia: 10000.0
```

21- Calcule e imprima la tabla de multiplicar del 9. Imprima el multiplicando el multiplicador y el producto.

Archivo de código (NetBeans) y Consola de salida (Ejercicio 19)



The screenshot displays the NetBeans IDE interface. The top pane shows the source code for a Java class named `Ejercicio21`. The code defines a package `ejercicio21` and a public class `Ejercicio21` with a `main` method. Inside the `main` method, an integer `multiplicando` is set to 9. A `for` loop iterates over the multiplier `multiplicador` from 1 to 10. For each iteration, the product is calculated as `multiplicando * multiplicador` and printed to the console using `System.out.println`. The bottom pane shows the output of the program, displaying the multiplication table for 9.

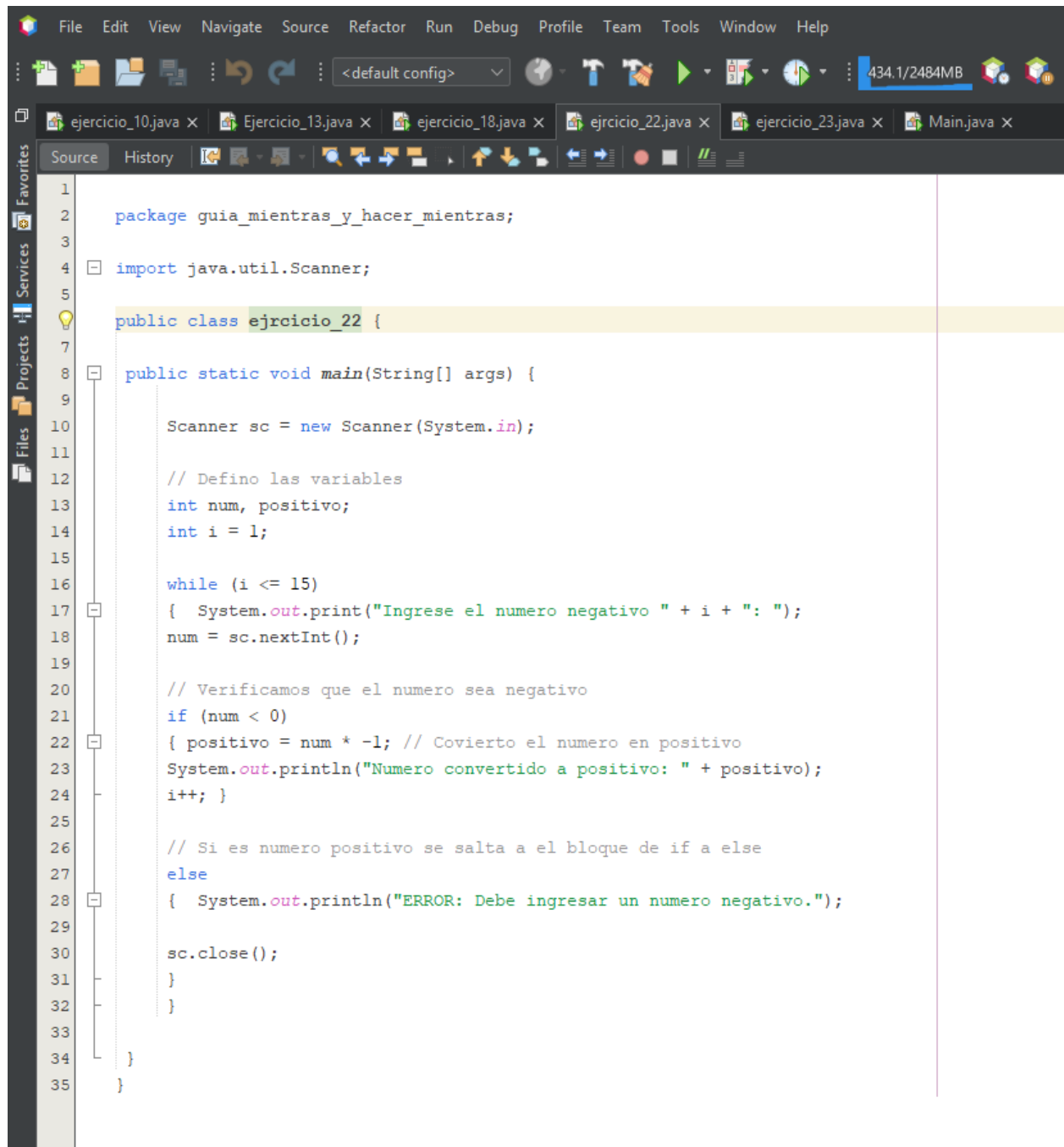
```
1 package ejercicio21;
2
3 public class Ejercicio21 {
4
5     public static void main(String[] args) {
6
7         int multiplicando = 9;
8
9         for (int multiplicador = 1; multiplicador <= 10; multiplicador++) {
10
11             int producto = multiplicando * multiplicador;
12
13             System.out.println(multiplicando + " x " + multiplicador + " = " + producto);
14         }
15     }
16 }
```

Output - ejercicio21 (run) x

```
run:
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

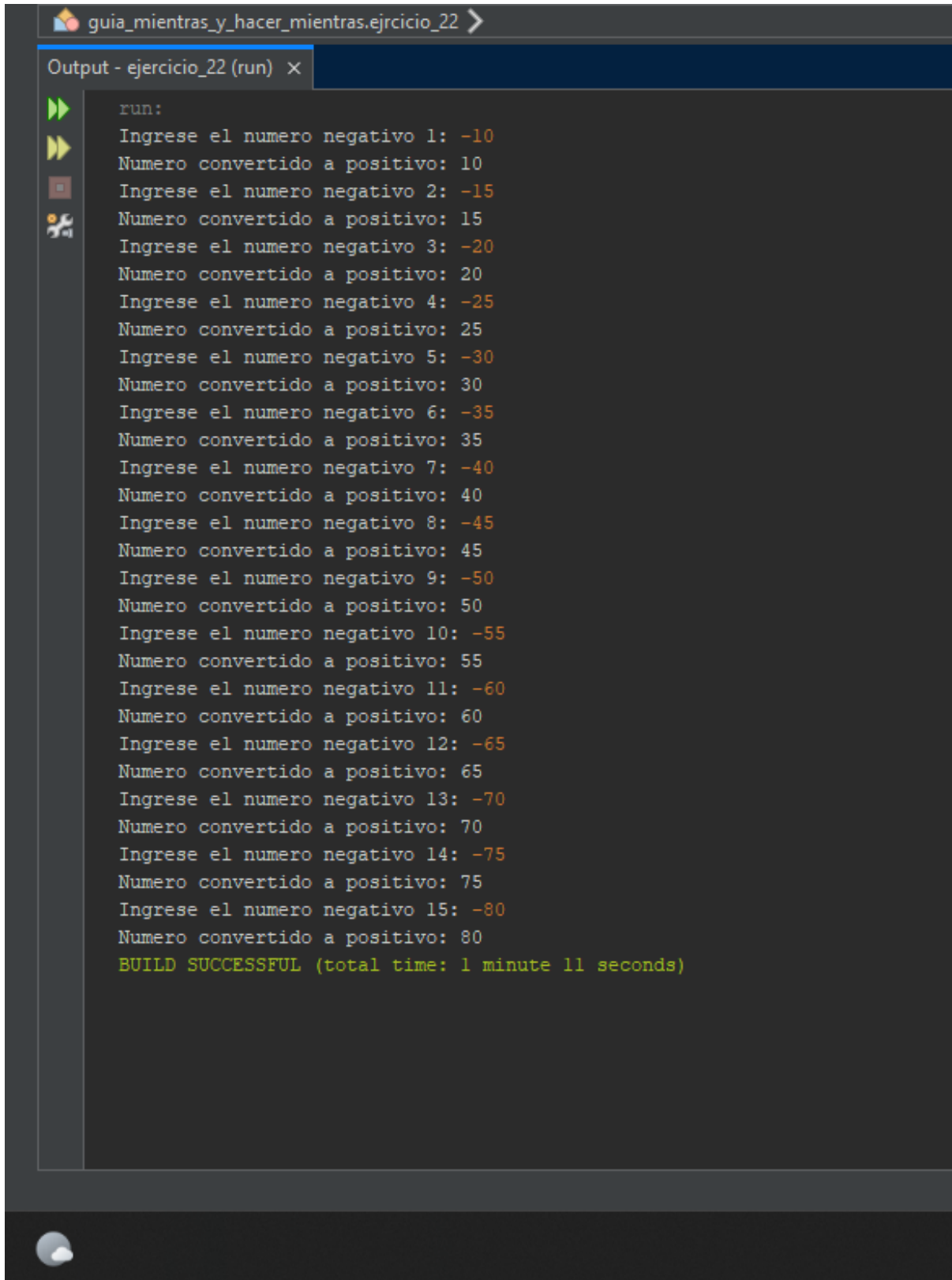
22- Leer 15 numero negativos y convertirlos a positivos e imprimir dichos números.

Archivo de código (NetBeans)



```
1 package guia_mientras_y_hacer_mientras;
2
3
4 import java.util.Scanner;
5
6 public class ejercicio_22 {
7
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11
12         // Defino las variables
13         int num, positivo;
14         int i = 1;
15
16         while (i <= 15)
17         { System.out.print("Ingrese el numero negativo " + i + ": ");
18           num = sc.nextInt();
19
20           // Verificamos que el numero sea negativo
21           if (num < 0)
22           { positivo = num * -1; // Covierto el numero en positivo
23             System.out.println("Numero convertido a positivo: " + positivo);
24             i++; }
25
26           // Si es numero positivo se salta a el bloque de if a else
27           else
28           { System.out.println("ERROR: Debe ingresar un numero negativo.");
29
30             sc.close();
31           }
32         }
33     }
34 }
35 }
```

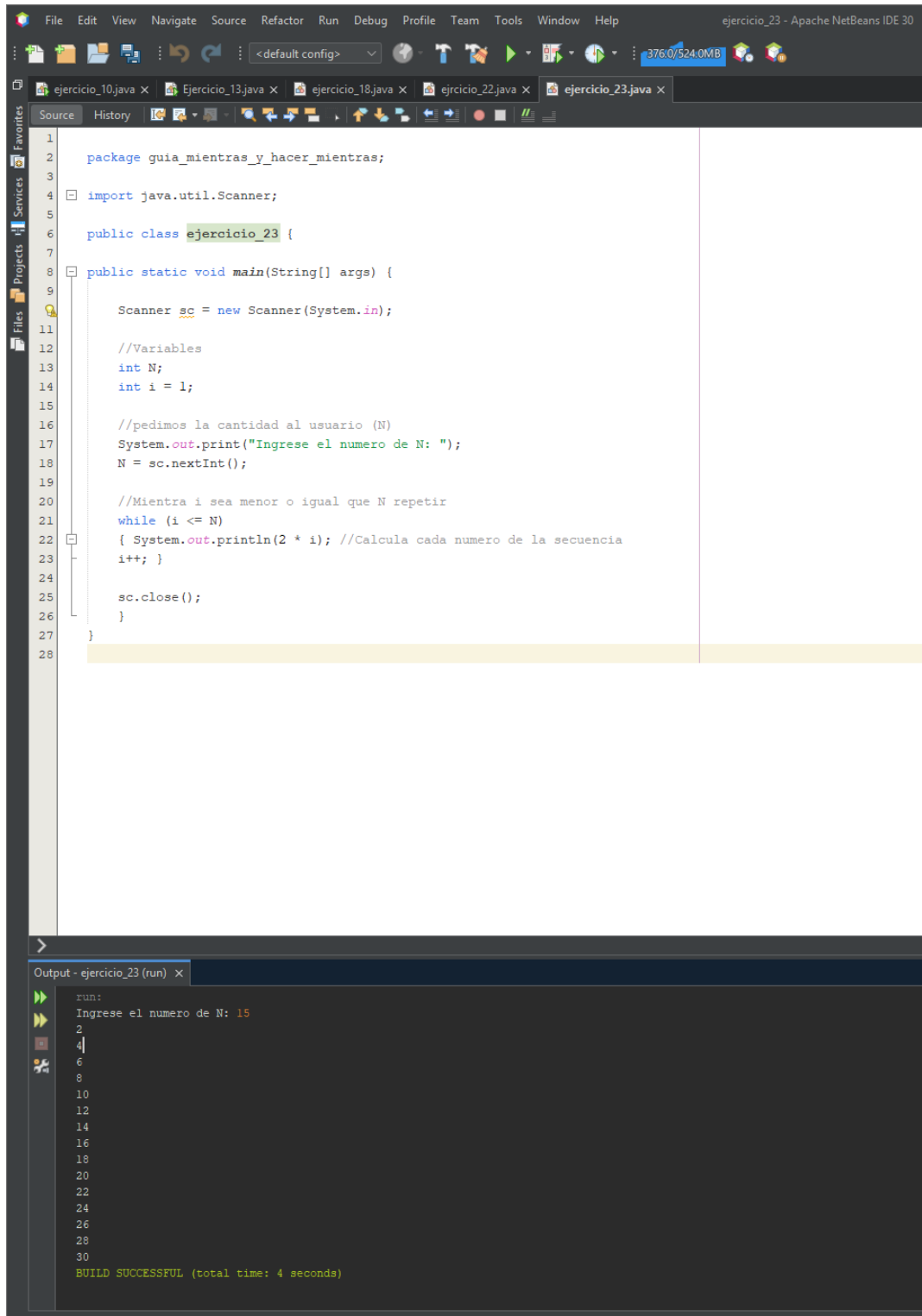
Consola de salida (Ejercicio 22)



```
run:
Ingrese el numero negativo 1: -10
Numero convertido a positivo: 10
Ingrese el numero negativo 2: -15
Numero convertido a positivo: 15
Ingrese el numero negativo 3: -20
Numero convertido a positivo: 20
Ingrese el numero negativo 4: -25
Numero convertido a positivo: 25
Ingrese el numero negativo 5: -30
Numero convertido a positivo: 30
Ingrese el numero negativo 6: -35
Numero convertido a positivo: 35
Ingrese el numero negativo 7: -40
Numero convertido a positivo: 40
Ingrese el numero negativo 8: -45
Numero convertido a positivo: 45
Ingrese el numero negativo 9: -50
Numero convertido a positivo: 50
Ingrese el numero negativo 10: -55
Numero convertido a positivo: 55
Ingrese el numero negativo 11: -60
Numero convertido a positivo: 60
Ingrese el numero negativo 12: -65
Numero convertido a positivo: 65
Ingrese el numero negativo 13: -70
Numero convertido a positivo: 70
Ingrese el numero negativo 14: -75
Numero convertido a positivo: 75
Ingrese el numero negativo 15: -80
Numero convertido a positivo: 80
BUILD SUCCESSFUL (total time: 1 minute 11 seconds)
```

23- Dado N números, calcular los números 2,4,6,8,..... 2N.

Archivo de código y consola de salida (Ejercicio 23)



The screenshot displays the Apache NetBeans IDE interface. The top menu bar includes File, Edit, View, Navigate, Source, Refactor, Run, Debug, Profile, Team, Tools, Window, and Help. The toolbar shows various icons for file operations, running, and debugging. The project explorer on the left shows the project structure with folders for Favorites, Services, Projects, and Files. The source editor displays the code for `ejercicio_23.java`.

```
1 package guia_mientras_y_hacer_mientras;
2
3
4 import java.util.Scanner;
5
6 public class ejercicio_23 {
7
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11
12         //Variables
13         int N;
14         int i = 1;
15
16         //pedimos la cantidad al usuario (N)
17         System.out.print("Ingrese el numero de N: ");
18         N = sc.nextInt();
19
20         //Mientras i sea menor o igual que N repetir
21         while (i <= N)
22         { System.out.println(2 * i); //Calcula cada numero de la secuencia
23         i++; }
24
25         sc.close();
26     }
27 }
28
```

The console output shows the execution of the program. It prompts the user to enter a number, and the user enters 15. The program then prints the sequence of even numbers from 2 to 30.

```
run:
Ingrese el numero de N: 15
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
BUILD SUCCESSFUL (total time: 4 seconds)
```